TUBAF
The University of Resources.
Since 1765.

PPP Final Report

Implementation of a Simulation Tool for the Gray–Scott Model

Final Report for the Personal Project Programming Module

Mehdi Bakhshi Zadeh

5 April 2026

TU Bergakademie Freiberg
Personal Project Programming

Supervisor: PD Dr. rer. nat. habil. Uwe Prüfert

Contents

List of Figures

List of Tables

Abstract

This project presents the development of a MATLAB simulation tool for the two-dimensional Gray–Scott reaction–diffusion model. The final program combines a shared numerical core with both non-GUI and GUI-based workflows, and includes multiple diffusion implementations, configurable boundary conditions, plotting and export utilities, reproducible verification tests, and benchmark scripts. The software was designed not only to produce Gray–Scott patterns, but also to provide a structured framework for testing solver correctness, comparing implementation variants, and supporting interactive exploration through an App Designer interface.

The numerical verification showed that the implementation reproduces the expected behavior of the underlying discretization methods. In particular, the manufactured-solution test confirmed second-order spatial accuracy of the discrete Laplacian, while the time-step study showed the expected first-order behavior of the explicit Euler scheme. Additional tests verified correct boundary-condition treatment, machine-precision agreement between matrix-based and stencil-based diffusion realizations, and consistency between GUI and non-GUI execution for the documented reference case. A Pearson-type pattern sanity check further supported the physical plausibility of the computed results.

A central practical objective of the project was to determine the most suitable solver configuration for the final implementation. The benchmark study showed that the dense formulation is mainly useful as a small-scale reference, whereas the sparse-matrix and stencil-based solvers are the practically relevant options. Among them, the stencil implementation showed the more favorable memory usage and scaling behavior over the relevant grid range, while remaining competitive in runtime. For this reason, the stencil solver was selected as the default solver in the final program. Overall, the project achieved its goal by delivering a verified, benchmarked, and usable Gray–Scott simulation tool with a modular structure suitable for further extensions.

1. Introduction

1.1. Motivation

Although the Gray–Scott model is compact in mathematical form, its numerical simulation becomes computationally demanding when spatial and temporal resolutions are increased in order to resolve pattern formation accurately. In particular, explicit time integration imposes a restrictive stability condition that links the admissible time step size to the spatial mesh width. As the grid is refined, the number of required time steps rises rapidly, which makes the model a useful test problem not only for reaction–diffusion dynamics, but also for the practical assessment of numerical efficiency.

A further motivation of this work is that the diffusion term can be implemented in more than one numerically equivalent way. On a structured Cartesian grid, the discrete Laplacian has a highly regular stencil structure. It can be realized explicitly through a sparse matrix construction, but it can also be applied in stencil form without assembling the full operator. This creates a natural computational question: which implementation is the more suitable practical solver for the Gray–Scott problem in the present MATLAB framework?

1.2. Objective

The objective of this project is to implement a MATLAB-based simulation tool for the two-dimensional Gray–Scott model and to determine, through systematic verification and benchmarking, which solver approach is the best practical choice for the underlying partial differential equation. The work therefore combines model implementation, numerical testing, and performance comparison in a single software framework.

A central requirement is that solver evaluation must be based on correctness first and efficiency second. For this reason, the project does not rely only on visually plausible solutions or isolated timing measurements. Instead, the implemented methods are verified for consistency and correctness before they are compared with respect to runtime and memory usage. The final aim is thus to provide a dependable simulation tool whose recommended default solver is justified by evidence rather than by assumption.

2. Theory and Mathematical Model

2.1. Gray–Scott Reaction–Diffusion System

Reaction–diffusion systems describe processes in which chemical species both react locally and spread in space by diffusion. Even relatively simple nonlinear interactions can generate complex spatial structures such as spots, stripes, travelling fronts, and labyrinth-like patterns. For this reason, reaction–diffusion equations are widely used as model systems for studying self-organization and pattern formation in chemistry, physics, and biology [1, 2].

The Gray–Scott model is a particularly well-known two-species reaction–diffusion system. It is based on an autocatalytic chemical mechanism in an open environment, where one component is continuously supplied and another is removed or decays. This continuous feeding and removal keep the system away from thermodynamic equilibrium and allow the formation of sustained non-uniform spatial patterns instead of simple relaxation to a homogeneous steady state [1–3].

In its standard form, the model uses two scalar concentration fields, denoted here by $u(\mathbf{x}, t)$ and $v(\mathbf{x}, t)$. Their dynamics are governed by the interaction of local nonlinear reaction kinetics and spatial diffusion. The central mechanism is cubic autocatalysis, meaning that the presence of one species promotes further production of itself through reaction with the other species. At the same time, diffusion tends to smooth spatial gradients. The patterns observed in the Gray–Scott system therefore arise from the competition between local nonlinear amplification and spatial spreading.

The Gray–Scott model is especially suitable for the present project because it combines a compact mathematical structure with rich qualitative behaviour. Small changes in the model parameters can lead to significantly different dynamical regimes, which makes the system an effective benchmark for the study of pattern-forming partial differential equations and their numerical simulation [1, 2].

2.2. Interpretation of the Model Terms

In the present work, the Gray–Scott model is written in the standard dimensionless form

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + F(1-u), \tag{2.1}$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (F+k)v, \tag{2.2}$$

where $u(\mathbf{x}, t)$ and $v(\mathbf{x}, t)$ denote the concentrations of the two interacting species, D_u and D_v are their diffusion coefficients, F is the feed rate, and k is the removal or decay rate of the second species [1, 2].

The diffusion terms, $D_u \nabla^2 u$ and $D_v \nabla^2 v$, describe spatial spreading from regions of higher concentration toward regions of lower concentration. If diffusion acted alone, both fields would tend to become spatially uniform. Pattern formation is therefore not caused by diffusion alone, but by its interaction with the nonlinear reaction kinetics.

The reaction term uv^2 represents the autocatalytic part of the dynamics. It consumes the species u and simultaneously produces the species v. This nonlinear coupling is the main local amplification mechanism in the model. Because the production rate depends quadratically on v, already existing amounts of v can enhance further growth of v, provided that enough u is available. This feedback is one of the main reasons why localized structures and self-replicating patterns can emerge.

The feed term $F(1-u)$ continuously drives the field u toward the reference value $u = 1$. Physically, this represents the external supply of one reactant from the surrounding environment. Without such forcing, the system would behave like a closed reaction system and would not sustain the same long-time pattern-forming behaviour. The term $(F+k)v$ in the second equation describes the loss of v through both outflow and decay. Together, the feed and removal terms make the model an open nonequilibrium system.

The qualitative behaviour of the Gray–Scott model is controlled mainly by the balance between diffusion, nonlinear autocatalytic reaction, feeding, and removal. In particular, the parameters F and k strongly influence whether the solution approaches a nearly homogeneous state or develops persistent heterogeneous patterns. For this reason, these parameters play a central role in the analysis of the model and in the numerical experiments carried out in this project [1, 2].

2.3. Domain, Initial Conditions, and Boundary Conditions

In this work, the Gray–Scott system is considered on a two-dimensional rectangular spatial domain

$$\Omega = [0, L_x) \times [0, L_y),$$

with concentration fields $u(\mathbf{x}, t)$ and $v(\mathbf{x}, t)$ defined over Ω. This setting is sufficiently simple to keep the model mathematically transparent, while still being rich enough to exhibit the characteristic spatial structures of the Gray–Scott dynamics [1, 2].

The initial state is chosen as a nearly homogeneous background together with a localized perturbation in the centre of the domain. In qualitative terms, the field u is initially close to its feed-dominated reference state, whereas v is initially close to zero except in a small central region where both concentrations are perturbed. Such a configuration is standard for Gray–Scott simulations because it provides a localized seed from which the subsequent pattern can develop. Small random perturbations are additionally useful for breaking exact symmetry and allowing the intrinsic dynamics of the system to select a spatial structure [1, 2].

For the main Gray–Scott study in this project, periodic boundary conditions are used as the default choice and are also used in the core tests. From a modelling perspective, periodic boundaries are natural for pattern-formation studies because they reduce artificial edge effects and allow the dynamics to be interpreted as evolution on a repeating domain. In this way, the observed structures are governed primarily by the competition between reaction and diffusion rather than by strong boundary constraints [1, 2].

For completeness, the implementation is not restricted to periodic boundaries only. It also supports Dirichlet and Neumann boundary conditions on each side of the domain. However, these alternatives are not the central focus of the present theory chapter. The main theoretical emphasis remains on the standard periodic Gray–Scott setting, since this is the reference configuration for the principal simulations and comparisons in the project.

2.4. Model Parameters Relevant to This Work

The central model parameters of the Gray–Scott system are the diffusion coefficients D_u and D_v together with the feed and removal parameters F and k. Among these, D_u and D_v control the relative spatial spreading of the two species, while F and k primarily determine the local reaction balance and therefore have a particularly strong influence on the qualitative behaviour of the solution. In practice, different combinations of these parameters can lead to markedly different dynamical regimes, ranging from almost homogeneous states to persistent heterogeneous patterns [1, 2].

In the present project, the diffusion coefficients are taken as

$$D_u = 2 \times 10^{-5}, \qquad D_v = 1 \times 10^{-5},$$

which means that the species u diffuses twice as fast as v. A representative default parameter choice used in the project is

$$F = 0.03, \qquad k = 0.06.$$

These values provide a convenient reference configuration for simulation and comparison, while remaining within the standard Gray–Scott setting considered in the literature.

At the same time, the project is not limited to a single fixed pair of (F, k) values. Instead, the Gray–Scott model is treated here as a parameter-dependent system whose behaviour can be explored over a broader range of feed and removal rates. This viewpoint is important because the main scientific interest lies not only in one isolated simulation, but in understanding how changes in the governing parameters affect the resulting patterns and long-time dynamics [1, 2].

Besides D_u, D_v, F, and k, the overall model setup also depends on the spatial domain size and on the form and strength of the initial perturbation. These quantities influence how patterns are triggered and how much spatial room is available for their development. However, from the theoretical perspective of the Gray–Scott system, the dominant control parameters remain F and k, since they most directly organize the transition between different qualitative regimes. For this reason, they play the most prominent role in the interpretation of the results presented later in this report.

3. Existing Code Resources and Project Originality

3.1. Closest Existing Code and Resources in the WWW

A focused research scan was carried out for publicly available repositories that directly implement the Gray–Scott reaction–diffusion model on a two-dimensional grid. This chapter is restricted to direct Gray–Scott repositories only. Therefore, general PDE frameworks and purely visual WebGL demonstrations are not discussed here. The closest resources found in the scan were compact MATLAB demonstration codes, educational multi-language implementations, and notebook-based Python projects. They were not complete MATLAB simulation frameworks of the kind developed in this project.

The closest MATLAB resource found is the repository by Sbalbi, which provides a MATLAB function for generating and recording Gray–Scott patterns. It allows the user to choose the feed and kill parameters and also exposes practical options such as grid size, initial state, colormap, and output file settings [4]. This makes it a useful and relevant reference for basic MATLAB-based Gray–Scott simulation and visualization. However, it is essentially a compact generator script rather than a larger project structure with multiple solver backends, automated testing, or systematic benchmarking.

Another close reference is the Gray–Scott material by Burkardt. His `gray_scott_pde` page provides a direct Gray–Scott solver and explicitly states that versions are available in MATLAB, Octave, and Python [5]. In addition, the related `gray_scott_movie` code focuses on generating graphics frames and assembling them into a movie [6]. These resources are especially relevant because they are direct, educational, and accessible implementations of the same model equation, but they are still organized as stand-alone numerical examples rather than as an integrated application with GUI control, verification scripts, and solver-comparison infrastructure.

The notebook-based project `Gray-Scott-PDE` by SABS-R3 is another close resource [7]. Its repository contains a Python package structure, a `tests` directory, and a main Jupyter notebook, and its README states that the time derivative is approximated by a forward difference while the Laplacian is approximated by a central difference scheme [7]. This makes it closer to a small teaching project than to a one-file demonstration. Even so, its workflow remains primarily notebook-driven and Python-based, whereas the present work develops a dedicated MATLAB implementation whose emphasis is not only

on producing patterns, but also on comparing solver realizations and documenting their behavior in a controlled benchmark setting.

Table 3.1 summarizes the closest direct Gray–Scott repositories identified in the research scan. The comparison shows that Gray–Scott implementations are widely available in the WWW, but they typically emphasize only one aspect at a time: a short MATLAB generator, an educational stand-alone solver, or a notebook-based Python project. In contrast, the present PPP combines numerical implementation, interface-level usability, verification, and benchmarking within a single MATLAB project. This does not make the Gray–Scott model itself original, but it does clearly define the practical gap in available public resources that this project addresses.

Resource	**Language(s)**	**Main focus**	**Main limitation relative to this PPP**
Sbalbi [4]	MATLAB	Compact Gray–Scott generator with recording and export options	Small demonstration code; no solver comparison, no automated tests, no project-scale structure
Burkardt [5, 6]	MATLAB, Octave, Python	Educational Gray–Scott solver and movie examples	Separate stand-alone examples rather than one integrated MATLAB application with GUI and benchmarking
SABS-R3 [7]	Python, Jupyter Notebook	Notebook-based teaching project with package structure and tests	Python-centered workflow; no MATLAB GUI and no solver-benchmark emphasis

Table 3.1.: Closest direct Gray–Scott repositories identified in the WWW research scan.

3.2. What This Project Implements Independently

The independent contribution of the present PPP is not the Gray–Scott model itself, since that model and many basic implementations already exist in the literature and in publicly available repositories. The contribution of this work is instead the development of a complete MATLAB project around the model, with emphasis on implementation quality, solver comparison, verification, and usability.

In contrast to short demonstration scripts, the present project is organized as a coherent codebase rather than as a single stand-alone file. The numerical model, parameter handling, simulation control, visualization workflow, and user interaction are connected through one consistent project structure. This makes the code suitable not only for producing Gray–Scott patterns, but also for studying how different implementation choices affect the behavior and efficiency of the simulation.

A central element implemented independently in this project is the use of more than one diffusion realization within the same MATLAB framework. In particular, the project supports different solver backends for the diffusion part, so that they can be compared under the

same model equations, parameter sets, and output conditions. This solver-comparison perspective is important for the PPP because benchmarking and implementation choice are among the main technical goals of the work, whereas the closest public repositories usually provide only one numerical realization and focus mainly on obtaining visual pattern formation.

Another independently implemented part is the integration of a graphical user interface with the simulation engine. The GUI is not a separate visual toy, but a front end connected to the same underlying model logic. This allows interactive exploration of presets and simulation settings while preserving a common numerical core. In addition, the project includes automated tests and dedicated benchmark scripts. As a result, the code is not only executable, but also testable and comparable in a systematic way.

For these reasons, the present PPP should be understood as an independently developed MATLAB simulation environment for the Gray–Scott system. Its originality lies in the structured implementation, the integration of solver variants, the connection between model and GUI, and the explicit verification and benchmarking workflow, rather than in claiming novelty for the underlying reaction–diffusion equations themselves.

3.3. Original, Adapted, and Adopted Parts

The present PPP does not claim originality for the Gray–Scott model itself, for the general idea of finite-difference discretization, or for the use of standard benchmark parameter sets from the literature. These elements belong to the established scientific and numerical background of the problem. Their role in this work is therefore adopted at the model and method level, not invented within the project.

At the code level, however, the implementation of the present PPP was developed independently. No external Gray–Scott repository was copied, and no source code fragment was directly adopted into the final MATLAB project. The publicly available resources discussed in the previous section were used only as part of the required research scan and as contextual references for comparison. Accordingly, the structure of the final codebase, the organization of the MATLAB files, the integration of the GUI with the numerical engine, the use of alternative solver backends within one framework, and the benchmark workflow are the result of the present project's own implementation work.

It is nevertheless important to distinguish independent implementation from complete conceptual isolation. The project naturally follows standard ideas that are common in numerical simulation software, such as separating model logic from visualization, using parameter presets, and checking numerical behavior by automated tests and benchmark scripts. In this sense, the work is methodologically informed by common scientific-computing practice, but it is not based on direct code reuse from the repositories identified in the WWW scan.

Therefore, the honest classification of the present PPP is as follows: the physical model and basic numerical principles are adopted from established literature, the overall software-engineering approach is independently implemented, and no direct source-code adaptation from the compared external Gray–Scott repositories is present in the final project. This distinction is important because it shows both academic honesty and the actual programming contribution of the work.

4. Numerical Methods Implementation

4.1. Overall Numerical Strategy, Computational Grid, and Data Layout

This chapter describes how the Gray–Scott reaction–diffusion model is solved numerically and how the final MATLAB implementation realizes the method in practice. The goal is not only to state the discrete equations, but also to explain how the final code organizes the solver, stores the state variables, applies the diffusion operator, and keeps the different solver variants consistent with each other. The final program is implemented in MATLAB R2025b and follows a deliberately modular design. The physical model is represented by two coupled scalar fields, $u(x, y, t)$ and $v(x, y, t)$, on a two-dimensional rectangular domain. In the code, the complete simulation pipeline is organized in the following sequence:

```
defaultParams → finalizeParams → buildGrid
makeOperator → GrayScottModel → eulerStep
```

This separation is important for both correctness and maintainability. Parameter definition, grid generation, operator construction, time stepping, plotting, and file export are not mixed in one monolithic script. Instead, each step has a well-defined responsibility, which makes the solver easier to verify, extend, and defend.

From the numerical point of view, the solver uses a method-of-lines style discretization. Space is discretized first on a uniform Cartesian grid, which transforms the partial differential equations into a coupled system of ordinary differential equations in time. These semi-discrete equations are then advanced by an explicit Euler method. Within each time step, the code computes the diffusion contribution, the nonlinear reaction contribution, optional source terms used for manufactured-solution tests, and finally the boundary-condition overwrite where required. This design keeps the mathematical structure of the Gray–Scott system visible in the implementation and makes the individual numerical ingredients easy to inspect separately.

The computational domain is defined as

$$\Omega = [0, L_x) \times [0, L_y),$$

with N_x grid points in the x-direction and N_y grid points in the y-direction. The corresponding uniform grid spacings are

$$h_x = \frac{L_x}{N_x}, \qquad h_y = \frac{L_y}{N_y}. \tag{4.1}$$

The grid points are therefore

$$x_i = ih_x, \qquad i = 0, 1, \ldots, N_x - 1,$$
$$y_j = jh_y, \qquad j = 0, 1, \ldots, N_y - 1.$$

The right endpoint is excluded in both directions. This is a deliberate and important implementation choice. For periodic boundary conditions, the points at $x = 0$ and $x = L_x$ represent the same physical location, and similarly for $y = 0$ and $y = L_y$. Including both endpoints would therefore duplicate one grid line and would make the periodic indexing inconsistent. The function `buildGrid.m` follows exactly this convention and constructs the grid on $[0, L_x) \times [0, L_y)$ rather than on $[0, L_x] \times [0, L_y]$.

In the final code, periodic boundaries are the default configuration, but the user may also select Dirichlet or Neumann conditions independently on each side of the domain. This flexibility is handled through the boundary-condition structure stored in `p.BC`. The important design choice is that the grid itself remains the same for all boundary types. In other words, the discretization does not change from one grid layout to another when the user switches boundary conditions. Instead, the solver keeps one consistent grid representation and applies the selected boundary treatment during the update process. This avoids unnecessary code duplication and keeps the main numerical kernel uniform across different simulation setups.

The state variables are stored in two complementary forms. For numerical operations that naturally act on the two-dimensional field, the concentrations are represented as arrays

$$U \in \mathbb{R}^{N_y \times N_x}, \qquad V \in \mathbb{R}^{N_y \times N_x}.$$

This is convenient for initialization, visualization, stencil-based diffusion, and explicit boundary updates. However, for matrix-based operator application and for a clean algebraic formulation, the same fields are also stored in vectorized form:

$$u = U(:), \qquad v = V(:).$$

The final solver therefore uses a deliberate dual representation: two-dimensional arrays for field-based logic and one-dimensional vectors for operator-based logic. This is one of the key implementation decisions of the project because it allows the sparse matrix, dense matrix, and matrix-free stencil realizations to share the same mathematical interpretation.

The vectorization follows MATLAB's column-major memory layout. If $U(j, i)$ denotes the field value at grid index (i, j), then the corresponding vector index is

$$k = j + (i - 1)N_y, \qquad i = 1, \ldots, N_x, \quad j = 1, \ldots, N_y. \tag{4.2}$$

This means that the y-index varies fastest in memory, while the x-index advances in blocks of size N_y. The choice is not arbitrary; it is exactly the storage convention used internally by MATLAB when a matrix is reshaped or flattened. As a result, the two representations are fully consistent:

$$u = U(:), \qquad U = \texttt{reshape}(u, N_y, N_x).$$

They do not require any manual permutation of indices. This consistency is essential for the correctness of the Kronecker-product Laplacian, because the matrix assembly must follow the same indexing convention as the state vectors used in time stepping.

The same storage convention also explains an important aspect of the implementation architecture. The sparse and dense operator backends apply the discrete Laplacian by matrix–vector multiplication. In contrast, the stencil backend reshapes the vector back into a two-dimensional field, evaluates neighbor contributions directly in array form, and then re-vectorizes the result. Since all three backends are tied to the same column-major mapping, they represent the same discrete spatial operator even though their internal realization differs. This common data layout is one of the reasons why equivalence tests between the matrix and stencil solvers can be performed in a clean and reliable way.

Another important practical aspect is the initialization of the state. The function `initialCondition.m` creates two $N_y \times N_x$ arrays, initially with $U = 1$ and $V = 0$, and then inserts a centered square perturbation region with modified concentrations. Small random perturbations can be added as well. Reproducibility is ensured through a user-defined random seed stored in the parameter structure and activated when the `GrayScottModel` object initializes or resets the state. This means that two runs with the same parameters and the same seed reproduce the same initial fields, which is essential for fair solver comparison, GUI consistency tests, and benchmark reproducibility.

The final code also caches the initial boundary values when a Dirichlet boundary is selected in `initial` mode. This is an implementation detail with clear numerical motivation. If a boundary is supposed to remain equal to the initial state, those values must be preserved explicitly rather than reconstructed later. The constructor of `GrayScottModel` therefore stores a boundary cache after initialization, so that subsequent time steps can re-impose the correct boundary values consistently. This is a good example of how the final software design supports the intended mathematical meaning of the boundary conditions.

At the highest level, the simulation state is encapsulated in the class `GrayScottModel`. This class stores the parameter structure, the grid, the chosen diffusion operator, the current state vectors, and the current simulation time and step counter. The actual one-step update is delegated to `eulerStep.m`, which receives the current vectors and returns the updated state plus diagnostic information. This separation is useful because it keeps the state management logic inside the model object while the numerical update rule remains transparent as a standalone function. The solver is not just a script that

repeatedly modifies variables, but a structured numerical program with a clear distinction between model state, operator construction, and one-step evolution.

In summary, the final implementation is built around three core ideas: a uniform Cartesian grid on $[0, L_x) \times [0, L_y)$, a column-major data layout consistent with MATLAB's native memory model, and a modular solver architecture that cleanly separates parameter handling, operator realization, and time stepping. These choices are not merely programming preferences; they directly support the numerical method, make the different solver variants comparable, and provide a robust foundation for the more detailed discretization and implementation discussions that follow in the next sections.

4.2. Spatial Discretization and the Discrete Laplacian

After defining the computational grid and the data layout, the next step is the spatial discretization of the diffusion term. In the Gray–Scott system, diffusion acts through the Laplacian operator on both concentration fields,

$$D_u \nabla^2 u, \qquad D_v \nabla^2 v.$$

The final solver approximates this operator on the uniform Cartesian grid introduced in the previous section by the standard second-order central finite-difference formula. This leads to the classical five-point stencil in two space dimensions.

Let $U(j, i)$ denote the grid value of a scalar field at the point (x_i, y_j), with

$$i = 0, 1, \ldots, N_x - 1, \qquad j = 0, 1, \ldots, N_y - 1.$$

For a uniform grid, the second derivative in the x-direction is approximated by

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) \;\approx\; \frac{U(j, i+1) - 2U(j, i) + U(j, i-1)}{h_x^2}, \tag{4.3}$$

and similarly in the y-direction,

$$\frac{\partial^2 u}{\partial y^2}(x_i, y_j) \;\approx\; \frac{U(j+1, i) - 2U(j, i) + U(j-1, i)}{h_y^2}. \tag{4.4}$$

Adding these two contributions gives the discrete Laplacian

$$(\Delta_h U)(j, i) = \frac{U(j, i+1) - 2U(j, i) + U(j, i-1)}{h_x^2} + \frac{U(j+1, i) - 2U(j, i) + U(j-1, i)}{h_y^2}. \tag{4.5}$$

This expression corresponds to the standard five-point finite-difference stencil on a two-dimensional Cartesian grid. The local coupling pattern of this operator is shown in Figure 4.1. On a uniform grid, the approximation is second-order accurate in space,

provided that the continuous solution is sufficiently smooth. Since the final code also supports rectangular domains with different mesh spacings in the two coordinate directions, the implementation retains the factors $1/h_x^2$ and $1/h_y^2$ separately rather than assuming $h_x = h_y$.

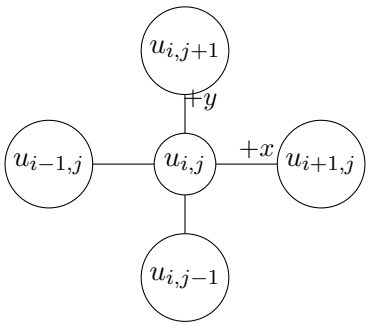

Figure 4.1.: Five-point finite-difference stencil used for the discrete Laplacian on the two-dimensional Cartesian grid.

The physical interpretation of (4.5) is straightforward. The value at each interior node is compared with its four nearest neighbors: east, west, north, and south. If the center value is larger than the neighborhood average, diffusion produces a negative contribution; if it is smaller, diffusion produces a positive contribution. In this sense, the discrete Laplacian measures local curvature or local imbalance, and the diffusion term acts to smooth the concentration field over time.

For periodic boundaries, the neighbor indices wrap around at the domain edges. Thus the point left of $i = 0$ is identified with $i = N_x - 1$, and the point right of $i = N_x - 1$ is identified with $i = 0$. The same applies in the y-direction. In the code, this periodic connectivity is built directly into the diffusion operator itself. This is a central design decision of the project: the Laplacian assembly and stencil application always use periodic connectivity as their internal baseline, while non-periodic boundary conditions are enforced afterward by explicitly overwriting boundary nodes. The boundary treatment will be discussed in detail in a later section; for the present section, the important point is that the spatial operator itself is defined in one consistent periodic form.

The final code realizes the same discrete Laplacian in three mathematically equivalent ways. The first realization is a sparse matrix form, the second is a dense matrix form, and the third is a matrix-free stencil form. Although these implementations differ in storage and application strategy, they all approximate the same operator (4.5). This separation between mathematical discretization and software realization is one of the most important ideas of the project.

To derive the matrix form, one first introduces the one-dimensional periodic second-derivative matrix. For a grid with N points, the unscaled operator has the tridiagonal

pattern

$$L_{1\mathrm{D}} = \begin{bmatrix} -2 & 1 & 0 & \cdots & 0 & 1 \\ 1 & -2 & 1 & \ddots & & 0 \\ 0 & 1 & -2 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 1 & 0 \\ 0 & & \ddots & 1 & -2 & 1 \\ 1 & 0 & \cdots & 0 & 1 & -2 \end{bmatrix}, \tag{4.6}$$

where the corner entries implement the wrap-around coupling required by periodicity. In the code, this operator is assembled by `buildLaplacian1D_periodic.m`. It returns the unscaled stencil matrix, and the factors $1/h_x^2$ and $1/h_y^2$ are introduced only when the full two-dimensional operator is built.

Because the state is vectorized in MATLAB column-major order as $u = U(:)$, the two-dimensional Laplacian is assembled by a Kronecker sum:

$$L = \frac{1}{h_y^2}\left(I_x \otimes L_y\right) + \frac{1}{h_x^2}\left(L_x \otimes I_y\right), \tag{4.7}$$

where L_x and L_y are the one-dimensional periodic second-derivative matrices in the x- and y-directions, and I_x, I_y are identity matrices of matching sizes. This formula is implemented in `buildLaplacian2D.m`. The ordering in (4.7) is not arbitrary. It is dictated by the mapping

$$u = U(:),$$

which stacks matrix columns one after another in MATLAB memory order. The term

$$\frac{1}{h_y^2}(I_x \otimes L_y)$$

acts along the rows of each column, while

$$\frac{1}{h_x^2}(L_x \otimes I_y)$$

couples corresponding entries across neighboring columns. If the ordering were inconsistent with the vectorization convention, the operator would connect the wrong nodes and the simulation would become physically meaningless. For this reason, the Kronecker structure and the storage layout must always be explained together.

The sparse matrix realization is used when `p.diffusionMode = "matrix"`. In that case, the operator struct contains the field `op.L`, and the diffusion contribution is computed by direct sparse matrix–vector multiplication:

$$\Delta_h u \approx Lu.$$

For the dense realization, selected by `p.diffusionMode = "full"`, the same operator is stored as a full matrix and applied in the same algebraic way. Numerically, both

versions represent the same discrete stencil. The difference lies only in storage format and computational cost. The dense realization is therefore not intended for large practical runs, but it remains useful as a reference implementation on small grids because it exposes the full operator explicitly.

The matrix-free stencil realization is selected by `p.diffusionMode = "stencil"`, which is the default mode in the final version of the code. In this case, the solver does not store the global matrix L. Instead, it stores only the minimal geometric information needed to evaluate the stencil, namely N_x, N_y, $1/h_x^2$, and $1/h_y^2$. These quantities are built once in `buildStencil2D.m` and stored in the struct `op.S`. During each time step, `applyLaplacianStencil.m` reshapes the vector u into the array U, constructs the east, west, north, and south neighbor arrays by circular shifts, evaluates the five-point formula directly, and finally vectorizes the result again:

$$u \;\to\; U \;\to\; \Delta_h U \;\to\; (\Delta_h U)(:).$$

This approach is mathematically equivalent to matrix multiplication by L, but it avoids storing the large global Laplacian explicitly.

The stencil implementation in the final code can therefore be interpreted as a matrix-free realization of exactly the same finite-difference operator. This point is important: the stencil backend is not a different numerical method, and it does not change the discrete model. It is simply a different implementation of the same second-order five-point Laplacian. The project does not compare different spatial discretizations against each other; it compares different realizations of one and the same discretization.

A practical advantage of the stencil form is that its memory requirements are determined mainly by the solution vectors and a very small operator description, rather than by storing the entire sparse or dense matrix. For this reason, the stencil realization is the most suitable default choice for larger problem sizes in the final code. The benchmark chapter later provides the quantitative evidence for this design decision. Here, it is enough to note that the matrix-free stencil keeps the mathematical discretization unchanged while reducing storage overhead in practical runs.

The project also includes internal checks that support the correctness of the discrete Laplacian. Since the five-point stencil is a consistent discretization of the continuous Laplacian, constant fields should produce a zero diffusion contribution. Moreover, the sparse periodic Laplacian has the expected symmetric structure, and the stencil and matrix applications should agree to machine precision when evaluated on the same state. These properties are verified in dedicated test files, including `test_laplacian_constant.m`, `test_laplacian_symmetry.m`, and `test_laplacian_stencil_equivalence.m`. Their role is not to introduce a new numerical method, but to confirm that the implementation matches the intended discrete operator.

In summary, the spatial discretization of diffusion in the final project is based on a standard second-order five-point finite-difference Laplacian on a uniform Cartesian grid. The corresponding discrete operator is realized either as a sparse matrix, a dense matrix,

or a matrix-free stencil, but all three forms are algebraically consistent with the same formula. This gives the project both numerical clarity and implementation flexibility: the mathematics remains fixed, while the software can choose the most suitable operator realization for the intended task.

4.3. Time Integration and the Full Update Algorithm

After spatial discretization, the Gray–Scott system is reduced to a large coupled system of ordinary differential equations in time. Let $u(t)$ and $v(t)$ denote the vectorized concentration fields after spatial discretization. Then the semi-discrete system can be written in the form

$$\frac{du}{dt} = D_u L u + f_u(u, v), \qquad \frac{dv}{dt} = D_v L v + f_v(u, v), \tag{4.8}$$

where L denotes the discrete Laplacian introduced in the previous section and

$$f_u(u, v) = -u \circ v^2 + F(1 - u), \qquad f_v(u, v) = u \circ v^2 - (F + k)v. \tag{4.9}$$

Here, $\circ$ denotes componentwise multiplication. The nonlinear reaction term is therefore evaluated pointwise at each grid node, whereas the diffusion term couples neighboring nodes through the discrete Laplacian.

In the final implementation, time integration is performed by the explicit forward Euler method. If u^n and v^n denote the numerical solution at time

$$t_n = n\,\Delta t,$$

then one step of the method is

$$u^{n+1} = u^n + \Delta t \left(D_u L u^n + f_u(u^n, v^n)\right), \tag{4.10}$$

$$v^{n+1} = v^n + \Delta t \left(D_v L v^n + f_v(u^n, v^n)\right). \tag{4.11}$$

This scheme is first-order accurate in time [8]. Its main advantage in the present project is its simplicity and transparency. Since the focus of the work is on the implementation and comparison of different diffusion backends, a fully explicit update makes the numerical workflow easy to trace and keeps the effect of each implementation choice clearly visible.

The explicit Euler step is implemented in the file `src/core/eulerStep.m`. Its structure follows the mathematical decomposition of the Gray–Scott model very closely. For each time step, the solver computes the diffusion contribution, then the reaction contribution, optionally adds source terms for manufactured-solution verification, advances the state by one explicit step, and finally enforces the selected boundary conditions. In abstract notation, the numerical update used in the code can be written as

$$u^{n+1} = \mathcal{B}(u^n + \Delta t\,[d_u^n + r_u^n + s_u^n]), \tag{4.12}$$

$$v^{n+1} = \mathcal{B}(v^n + \Delta t\,[d_v^n + r_v^n + s_v^n])\,, \tag{4.13}$$

where d_u^n, d_v^n are the diffusion contributions, r_u^n, r_v^n are the reaction contributions, s_u^n, s_v^n are optional source terms, and $\mathcal{B}$ denotes the boundary overwrite operator applied after the explicit update. When no manufactured source is used, the source vectors are simply zero.

The reaction part is implemented in `src/reaction/reactionGrayScott.m`. There the nonlinear product

$$u \circ v^2$$

is evaluated componentwise, and the reaction vectors are formed exactly according to (4.9). Since this computation acts locally at each grid node, it does not depend on the chosen diffusion backend. This is an important structural property of the final code: the diffusion realization may change between matrix, dense, and stencil modes, but the reaction update remains identical in all cases.

The diffusion contribution is computed by `src/core/applyDiffusion.m`. This function dispatches to the appropriate backend according to the parameter

$$\texttt{p.diffusionMode} \in \{\texttt{"matrix"}, \texttt{"full"}, \texttt{"stencil"}\}.$$

Thus, for each time step, the code evaluates

$$d_u^n = D_u \Delta_h u^n, \qquad d_v^n = D_v \Delta_h v^n,$$

but the internal realization of Δ_h depends on the selected operator backend. The time integrator itself does not need to know whether the Laplacian is being applied by sparse matrix multiplication, dense matrix multiplication, or a matrix-free stencil. This is one of the cleanest aspects of the implementation: the one-step update routine is written once, while the diffusion backend is exchanged through a common interface.

In addition to the physical Gray–Scott terms, the final implementation supports optional source terms through the user-defined field `p.sourceFcn`. This functionality is used for the manufactured-solution tests included in the verification part of the project. If such a source function is present, the solver evaluates

$$(s_u^n, s_v^n) = s(x, y, t_n, p)$$

on the current grid and adds the resulting vectors inside the Euler update. If no source function is defined, the source contribution is set to zero automatically. This design allows the same stepping routine to be used both for ordinary simulation and for verification problems, without modifying the core algorithm.

The temporal update is therefore fully explicit. At each step, all right-hand-side terms are evaluated using the old state (u^n, v^n), and the new state is obtained directly by addition. No nonlinear solve, linear solve, or iterative correction is required. From an implementation perspective, this keeps the code compact and makes the relation between the discrete equations and the program logic very direct.

The price of this simplicity is the usual stability restriction of explicit methods. For the pure diffusion equation discretized by central differences in space and forward Euler in time, a standard sufficient stability condition is

$$\Delta t \leq \frac{1}{2D\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right)}, \tag{4.14}$$

where D denotes the diffusion coefficient [8]. In the special case $h_x = h_y = h$, this reduces to

$$\Delta t \leq \frac{h^2}{4D}. \tag{4.15}$$

Since the Gray–Scott model contains two diffusing species, the timestep must satisfy this restriction for both D_u and D_v. A practical sufficient choice is therefore

$$\Delta t \leq \min\left\{\frac{1}{2D_u\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right)}, \frac{1}{2D_v\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right)}\right\}. \tag{4.16}$$

For the full nonlinear Gray–Scott system, this should be interpreted as a diffusion-based practical restriction rather than a complete sharp stability bound, since the reaction terms also influence the timestep limitations. As a result, the admissible timestep becomes more restrictive when the grid is refined, because the discrete diffusion operator scales with $1/h_x^2$ and $1/h_y^2$ [8]. In practical terms, a smaller mesh spacing improves spatial resolution but reduces the stable explicit timestep. The final project therefore uses conservative timestep choices in the default parameter sets, and timestep-related behavior is checked separately in dedicated verification tests. The detailed quantitative comparison of timestep effects belongs to the verification and benchmark discussion rather than to the present implementation chapter.

Another relevant implementation detail is the distinction between simulation time and step count. The model class stores both quantities explicitly through the fields

`obj.t` and `obj.n`.

After every successful call to the stepping routine, the time is updated according to

$$t \leftarrow t + \Delta t,$$

and the step counter is incremented by one. This provides a clean separation between physical time and iteration number. It also allows plotting and export routines to be triggered at user-defined step intervals such as every N steps, independently of the total final time.

At the level of the `GrayScottModel` class, one complete step is performed by the method `step()`, which calls `eulerStep.m`, stores the returned solution vectors, and

updates the internal counters. The high-level model object therefore manages the simulation state, while the standalone stepping routine contains the actual numerical formula. This separation keeps the time integrator transparent and avoids duplicating step logic in scripts, tests, and the GUI controller.

The one-step function also returns a diagnostic structure. In the final implementation, this structure contains the minimum and maximum values of both fields after the update, together with a flag indicating whether any `NaN` or `Inf` value has occurred. These diagnostics do not modify the method itself, but they make the simulation safer and easier to monitor. Detecting non-finite values immediately is especially useful when parameters or time-step settings are changed during testing.

From an algorithmic viewpoint, the complete update sequence used in one time step can be summarized as follows:

1. Start from the current vectors u^n and v^n.
2. Compute the diffusion contributions with the selected backend.
3. Compute the nonlinear reaction contributions pointwise.
4. Add optional source terms if a manufactured-solution test is active.
5. Form the explicit Euler update for both fields.
6. Overwrite boundary nodes according to the selected boundary-condition specification.
7. Compute diagnostics and advance the stored simulation time and step counter.

This sequence reflects the actual structure of the final code and provides a direct bridge between the semi-discrete equations and the software implementation.

In summary, the time integration in the final project is based on a first-order explicit Euler method applied to the semi-discrete Gray–Scott system. The method is simple, fully transparent, and implemented in a modular way: diffusion, reaction, optional source terms, boundary enforcement, and diagnostics are handled in separate components but combined in one consistent update routine. This design keeps the numerical method clear while allowing the implementation to support verification features and multiple diffusion backends without changing the time-stepping logic.

4.4. Boundary-Condition Treatment

In the final implementation, boundary conditions are handled in a way that keeps the diffusion backend, the time integrator, and the user interface consistent with each other.

The code supports boundary conditions independently on the four sides of the rectangular domain,

$$\texttt{left}, \quad \texttt{right}, \quad \texttt{bottom}, \quad \texttt{top},$$

through the parameter structure `p.BC`. For each side, the user may select one of the three types

$$\texttt{periodic}, \quad \texttt{dirichlet}, \quad \texttt{neumann}.$$

Periodic boundaries are the default configuration, but Dirichlet and Neumann conditions can be chosen separately on each side. This design makes the solver flexible without changing the underlying grid or the main time-stepping routine.

The boundary-condition handling is normalized in `src/core/finalizeParams.m`. If a boundary specification is missing, it is completed automatically with safe defaults. In particular, all four sides are initialized as periodic, the default Dirichlet mode is `initial`, and the default Neumann mode is `constant`. This normalization step ensures that the subsequent numerical routines always receive a complete and validated boundary-condition structure.

The essential implementation idea is that the discrete diffusion operator itself is built with periodic connectivity, while non-periodic boundary conditions are enforced afterward by explicit overwrite of the boundary nodes. Thus, the diffusion operator remains the same mathematical object across all solver backends, and boundary handling is treated as a separate layer in the update process. This choice is especially useful in a code base that supports sparse-matrix, dense-matrix, and matrix-free stencil realizations of the same Laplacian, because it avoids having to build a different operator for every possible combination of side conditions.

After one explicit Euler step, `applyBoundaryConditions.m` reshapes the state vectors into two-dimensional arrays:

$$U = \mathrm{reshape}(u, N_y, N_x), \qquad V = \mathrm{reshape}(v, N_y, N_x),$$

and then overwrites the relevant boundary entries according to the selected side conditions. The side conventions used throughout the code are

$$\texttt{left} : U(:, 1), \qquad \texttt{right} : U(:, N_x),$$

$$\texttt{bottom} : U(1, :), \qquad \texttt{top} : U(N_y, :),$$

with the same layout used for the field V. This explicit convention is important because all boundary operations, tests, and GUI selections rely on the same indexing scheme.

If a side is periodic, no overwrite is performed. The periodic coupling is already built into the discrete Laplacian through the wrap-around structure of the operator. In other words, for periodic boundaries the diffusion step itself already uses the correct neighboring values across the domain edges, so no additional enforcement is necessary after the time update.

For Dirichlet boundaries, the boundary values are imposed strongly by direct assignment after each explicit time step. If the prescribed boundary value is denoted by g, then the code sets

$$U(:,1) = g_{\text{left}}, \qquad U(:,N_x) = g_{\text{right}}, \tag{4.17}$$

for left and right boundaries, and

$$U(1,:) = g_{\text{bottom}}, \qquad U(N_y,:) = g_{\text{top}}. \tag{4.18}$$

The same overwrite is applied separately to the V field, so the concentrations of the two species may be prescribed independently. This is the standard finite-difference treatment of Dirichlet conditions, in which the boundary node values are set directly to the prescribed data [8]. In the final code, Dirichlet boundaries support two modes. In `constant` mode, the user provides fixed values through

`p.BC.<side>.dirichletValue.u`, `p.BC.<side>.dirichletValue.v`.

In `initial` mode, the corresponding boundary values are frozen from the initial condition and then reused at every subsequent step. This is implemented through a boundary cache, `p.BCcache`, which is created when the model is initialized or reset. As a result, the code can preserve initial edge values exactly without recomputing them later.

For Neumann boundaries, the implementation uses first-order one-sided finite differences [8]. If the outward normal derivative is prescribed as

$$\frac{\partial u}{\partial n} = g,$$

then the boundary value is reconstructed from the adjacent interior value. On the left boundary, the discrete condition is

$$\frac{U(:,2) - U(:,1)}{h_x} = g_{\text{left}}, \tag{4.19}$$

which gives

$$U(:,1) = U(:,2) - h_x g_{\text{left}}. \tag{4.20}$$

Similarly, on the right boundary,

$$\frac{U(:,N_x) - U(:,N_x-1)}{h_x} = g_{\text{right}}, \tag{4.21}$$

so that

$$U(:,N_x) = U(:,N_x-1) + h_x g_{\text{right}}. \tag{4.22}$$

For the bottom and top boundaries, the corresponding formulas are

$$U(1,:) = U(2,:) - h_y g_{\text{bottom}}, \qquad U(N_y,:) = U(N_y-1,:) + h_y g_{\text{top}}. \tag{4.23}$$

Again, the same construction is applied independently to the V field. In the present implementation, Neumann boundaries are supported in `constant` mode, meaning that the user prescribes constant normal derivatives on each side.

This boundary treatment is integrated into the one-step update described in the previous section. If $\widetilde{u}^{n+1}$ and $\widetilde{v}^{n+1}$ denote the provisional Euler update before boundary enforcement, then the final stored state is

$$u^{n+1} = \mathcal{B}\big(\widetilde{u}^{n+1}\big)\,, \qquad v^{n+1} = \mathcal{B}\big(\widetilde{v}^{n+1}\big)\,, \tag{4.24}$$

where $\mathcal{B}$ denotes the overwrite operation defined by the selected side conditions. In this way, boundary enforcement is performed strongly after every time step. The update does not rely on weak penalization or embedded ghost-cell iterations; instead, the prescribed boundary behavior is imposed directly on the stored state arrays.

A practical consequence of this design is that all diffusion backends remain compatible with the same boundary-condition routine. The sparse matrix mode, the dense matrix mode, and the stencil mode all produce a provisional diffusion update using the same periodic baseline operator. Boundary conditions are then enforced in exactly the same way, independently of how the Laplacian was applied. This is one of the main reasons why matrix and stencil solutions can be compared cleanly: the backend changes only the realization of diffusion, not the surrounding enforcement logic.

The implementation also supports mixed boundary configurations. For example, one side may be Dirichlet, another Neumann, and the remaining sides periodic. This capability is verified in `tests/test_boundary_conditions.m`, where constant Dirichlet enforcement, Neumann derivative enforcement, and mixed boundary configurations are checked across all three diffusion modes. These tests confirm that the same boundary rules are applied consistently in the matrix, stencil, and dense implementations.

From a numerical perspective, the chosen approach is primarily an implementation strategy rather than a change of the underlying model. Periodic boundaries are incorporated directly into the operator, while Dirichlet and Neumann conditions are imposed by explicit update of the edge nodes after each step. This gives the project a simple and robust boundary framework: the operator construction remains uniform, the user can switch boundary types per side, and the final stored state always satisfies the selected edge conditions exactly at the discrete level.

In summary, the final code uses a hybrid boundary-handling design. Periodic boundaries are encoded directly in the discrete Laplacian, whereas Dirichlet and Neumann conditions are enforced strongly by post-step overwrite on the two-dimensional state arrays. The result is a boundary-condition implementation that is flexible, backend-independent, and consistent with the modular structure of the solver.

4.5. Diffusion Backends and Default Solver Choice

The project supports three realizations of the discrete diffusion operator, selected through the parameter

$$\texttt{p.diffusionMode} \in \{\texttt{"matrix"}, \texttt{"full"}, \texttt{"stencil"}\}.$$

All three options represent the same second-order finite-difference Laplacian introduced in Section 4.2. They do not change the mathematical model, the computational grid, or the time integrator. Instead, they differ only in how the discrete operator is stored and applied in the code. This distinction is central to the project, because the main implementation question is not which diffusion model to use, but which realization of the same model is most suitable in practice.

The construction of the operator is handled by `src/core/makeOperator.m`. This function inspects the selected diffusion mode and prepares a backend-specific operator structure before the time loop begins. As a result, the cost of building the operator is paid only once during initialization, rather than being repeated at every time step. This is an important implementation choice, since the diffusion operator depends only on the grid and the selected backend, not on the evolving concentrations themselves. Preconstructing the operator therefore avoids unnecessary repeated work during simulation.

If the mode is set to `matrix`, the function `src/operators/buildLaplacian2D.m` assembles the two-dimensional periodic Laplacian as a sparse matrix. This matrix is stored in the field `op.L`, and the diffusion contribution is evaluated by sparse matrix–vector multiplication. The sparse backend is algebraically explicit and closely reflects the Kronecker-sum formulation of the operator. It is therefore convenient for verification, inspection, and direct comparison with the other backends. At the same time, it requires storage not only for the numerical values of the operator, but also for the sparse index structure used by MATLAB to represent the matrix efficiently.

If the mode is set to `full`, the same operator is converted to a dense matrix representation and stored explicitly. The diffusion term is then evaluated by ordinary dense matrix–vector multiplication. This mode is not intended for large-scale simulation, because a dense matrix for a two-dimensional Laplacian quickly becomes prohibitively expensive in memory. Its main role in the final code is therefore as a small-grid reference realization. Since the operator is stored completely and explicitly, it can be useful for debugging, sanity checks, and direct comparison on small problem sizes. For serious production runs, however, the dense backend is not the preferred choice.

If the mode is set to `stencil`, no global Laplacian matrix is stored. Instead, the code builds a compact stencil description in `buildStencil2D.m`. This structure contains only the information needed to evaluate the five-point stencil directly on the grid. During each step, `applyDiffusion.m` calls `applyLaplacianStencil.m`. That routine reshapes the vectorized state into a two-dimensional array, forms the neighboring values by periodic shifts, evaluates the stencil locally, and then vectorizes the result again. This backend is

therefore matrix-free: it applies the same discrete operator without assembling the full global matrix.

The three backends can be summarized conceptually as follows:

- `matrix`: sparse global operator, applied by sparse matrix–vector multiplication;
- `full`: dense global operator, applied by dense matrix–vector multiplication;
- `stencil`: matrix-free local operator evaluation on the grid.

Although these realizations are different computationally, they remain mathematically equivalent at the discrete level. The same five-point Laplacian is used in all cases, and the same explicit Euler update is applied afterward. This makes it possible to compare the backends fairly, since any observed difference comes from the implementation strategy rather than from a change in the numerical scheme.

A major advantage of the final architecture is that all three backends are accessed through the same interface. The time-stepping routine `src/core/eulerStep.m` does not contain separate logic for sparse, dense, and stencil diffusion. Instead, it calls the common helper `src/core/applyDiffusion.m`, which dispatches internally to the appropriate backend. This design keeps the step routine compact and avoids code duplication. It also improves maintainability: backend-specific changes remain localized in the operator layer and do not propagate through the rest of the solver.

The project includes dedicated consistency tests to verify that these backends produce the same discrete diffusion action up to machine precision when applied to the same state. In particular, the stencil and matrix realizations are checked against each other, and the resulting equivalence confirms that the stencil mode is not an approximation of lower quality, but simply a different realization of the same operator. This is a key point for the implementation logic of the project. The choice of backend affects memory use and computational cost, but not the underlying finite-difference discretization.

In the final version of the project, the stencil backend is selected as the default diffusion mode. This choice was made for practical computational reasons. The sparse-matrix realization already reduces storage compared with a dense matrix, but it still requires an explicitly assembled global operator together with its sparse indexing structure. The stencil realization avoids this global matrix storage entirely and therefore has lower structural overhead. Its memory footprint is determined mainly by the state vectors and a small set of geometric coefficients, rather than by the storage of the full operator itself.

This property becomes more important as the grid size increases. For a two-dimensional problem with

$$N = N_x N_y$$

degrees of freedom per species, a dense matrix representation scales like $O(N^2)$ in storage and is therefore unsuitable except for very small test cases. The sparse representation reduces this to $O(N)$ nonzero storage, but still carries the overhead associated with explicit matrix representation. The stencil realization also scales effectively with

$O(N)$ in the size of the evolving fields, but without storing the global Laplacian matrix. For this reason, it is the most natural choice when the objective is to keep the solver practical for larger grids while preserving the same discrete method.

The benchmark study presented later in the report provides the quantitative evidence for this decision. In that study, the stencil backend shows a more favorable practical memory behavior for larger problem sizes, and its measured memory use remains closer to the expected low-storage behavior of a matrix-free implementation. For the present chapter, it is sufficient to state the implementation consequence: the final code uses the stencil backend as the default solver because it preserves the same numerical discretization while reducing storage overhead in practical runs.

The existence of multiple backends is not only useful for performance considerations, but also for software reliability. The sparse matrix mode provides a clear algebraic reference implementation, the dense mode provides a fully explicit small-scale reference, and the stencil mode provides the preferred production backend. This layered design makes the code easier to verify. When two independently implemented realizations of the same operator agree numerically, confidence in the correctness of the solver increases significantly.

Another important implementation aspect is that the default backend choice is propagated consistently through the whole project. The default parameter set selects `stencil`, the main non-GUI runner uses the same default, and the graphical user interface is aligned with that choice as well. As a result, the user sees one coherent solver behavior across scripts, tests, and interactive runs. The backend is still switchable when comparison or debugging is needed, but the default configuration reflects the preferred practical implementation.

From a software-engineering perspective, this backend design captures one of the main contributions of the project. The solver is not restricted to a single hard-coded operator realization. Instead, it provides a common simulation framework in which multiple mathematically equivalent diffusion backends can be tested, validated, and compared under identical conditions. This makes the implementation both more flexible and more informative than a single-backend code base.

In summary, the final project implements three realizations of the same discrete Laplacian: a sparse matrix backend, a dense matrix backend, and a matrix-free stencil backend. Among these, the stencil backend is chosen as the default because it avoids storing the global diffusion matrix, reduces structural memory overhead, and remains fully consistent with the same finite-difference scheme used by the other realizations. The result is a solver framework that combines numerical consistency with practical flexibility and backend-aware implementation design.

4.6. MATLAB Implementation Architecture and Practical Optimizations

Beyond the numerical method itself, an important part of the project is the way the solver is organized as a MATLAB program. The final implementation is written in MATLAB R2025b and is structured as a modular code base rather than as a single monolithic script. This design choice improves readability, simplifies testing, and makes it possible to modify one component of the program without changing the rest of the solver. In particular, parameter definition, grid generation, diffusion-operator construction, reaction evaluation, one-step time integration, plotting, output export, and testing are separated into dedicated files with clearly defined responsibilities.

The code base is organized into several functional layers. The directory `src/core` contains the central numerical workflow, including parameter normalization, grid construction, diffusion dispatch, time stepping, initial conditions, and boundary enforcement. The directory `src/operators` contains the different realizations of the discrete Laplacian. The directory `src/reaction` contains the Gray–Scott reaction term, while `src/model` contains the class `GrayScottModel.m`, which encapsulates the complete simulation state. Input/output functionality, such as plotting and frame export, is placed in `src/io`. The GUI is stored separately in `scripts/GrayScottController.mlapp`, and the verification and benchmark routines are kept in the `tests` directory. This separation of concerns keeps the numerical core independent from the interface and from the testing framework.

At the entry-point level, the project provides two main launchers. The script `run_model_class.m` executes the simulation in a non-GUI mode and is suited for reproducible scripted runs, saved output, and batch-like testing. The script `run_gui.m` launches the graphical interface. Both entry points add the required source folders to the MATLAB path and then rely on the same underlying core functions. This is an important architectural decision: the GUI is not a separate solver, but only a controller layer that accesses the same numerical engine as the non-GUI execution path. As a result, scripted runs, tests, and interactive runs remain consistent with each other.

The parameter workflow is deliberately staged. The function `defaultParams.m` defines a complete parameter set with physically meaningful defaults, including grid size, diffusion coefficients, feed and kill parameters, timestep, final time, plotting cadence, file output cadence, initial-condition type, random seed, boundary-condition defaults, and the selected diffusion backend. After that, `finalizeParams.m` validates and normalizes the user-edited parameter structure. This second step is important because users or scripts may modify only part of the parameter set. The normalization step therefore guarantees that missing or incomplete settings are completed with valid defaults before the simulation begins. In particular, the diffusion mode is checked, and the boundary-condition structure is expanded into a complete four-sided representation.

The next architectural step is grid construction. The function `buildGrid.m` computes the mesh spacing, the coordinate vectors, and the two-dimensional mesh arrays associated with the problem domain. The grid object is then passed explicitly into the model constructor. This means that the model does not hide the domain geometry internally or rebuild it implicitly. Instead, the grid is treated as a first-class object in the simulation pipeline. This improves transparency and ensures that the diffusion operator and the state initialization are built on the same geometric information.

The central state container is the class `src/model/GrayScottModel.m`. This class stores the parameter structure, the grid, the selected diffusion operator, the current solution vectors, the current simulation time, and the current step index. The class-based design provides a clean separation between persistent simulation state and the numerical update logic. The state variables `u` and `v` are kept inside the object, while the actual update formula is implemented in the external function `eulerStep.m`. This avoids repeating state-management code in different scripts and makes the model behavior uniform across GUI runs, non-GUI runs, and tests.

The constructor of `GrayScottModel` performs three important tasks. First, it validates and stores the parameter structure. Second, it constructs the diffusion operator once through the backend-specific operator builder. Third, it initializes the concentration fields and stores them as vectorized state variables. This means that the costly operator setup is separated from the repeated time-stepping phase. Once the model object has been created, the simulation loop only applies the already constructed operator and advances the state. This is one of the main implementation optimizations of the project: objects that depend only on the grid and the chosen backend are created once and then reused throughout the simulation.

The model initialization also contains an explicit reproducibility measure. If the parameter structure includes a seed, the constructor uses

```
rng(p.seed,"twister")
```

before generating the initial condition. As a result, the random perturbations used in the baseline and Pearson-style initial conditions are reproducible across runs. This is essential for meaningful backend comparison, GUI/non-GUI consistency testing, and benchmark reproducibility. Without a controlled seed, two runs with identical nominal parameters could produce slightly different pattern development simply because the initial perturbation differed.

Another implementation detail with practical importance is the boundary cache used for Dirichlet conditions in `initial` mode. When the model is initialized, the code stores the initial values along the left, right, bottom, and top boundaries in the structure `p.BCcache`. These cached values are then reused whenever the corresponding boundary must remain equal to the initial state. This avoids recomputing or reconstructing those values later and guarantees that the Dirichlet condition remains exactly consistent with the initially generated fields. The cache therefore serves both correctness and clarity.

The one-step evolution itself is intentionally modular. The function `eulerStep.m` receives the current state vectors, the operator structure, the parameter struct, and the current time. It then computes diffusion, reaction, optional manufactured source terms, the explicit Euler update, and boundary enforcement. Since the diffusion contribution is obtained through the helper `applyDiffusion.m`, the time-stepper remains independent of the selected backend. This modularity has a direct software benefit: modifications to the operator implementation do not require changes to the reaction code or to the time integrator, as long as the common diffusion interface is preserved.

A related architectural strength of the project is its separation between numerical logic and visualization logic. The functions `plotState.m` and `exportFrame.m` are placed in `src/io` and do not modify the numerical solution. They operate only on already computed states. This separation is useful because it prevents plotting code from becoming entangled with the solver itself. The numerical result is therefore independent of whether live plotting or PNG export is enabled.

The implementation also contains several practical optimizations that improve runtime behavior or software robustness without changing the numerical method.

A first optimization is the reuse of graphics objects during live plotting. In `plotState.m`, persistent handles are used for the image and the colorbar. Instead of recreating the plot from scratch at every update, the code updates only the displayed image data through the `CData` property. This reduces repeated graphics-object creation and makes live visualization more efficient, especially for long runs with many output frames.

A closely related optimization is used in `exportFrame.m`. There, the code maintains a persistent invisible figure, axis, and image object. When a new snapshot is exported, the figure is not recreated; only the image content and title are updated before printing the result to PNG. This is more efficient and more robust than repeatedly opening and closing figures during a long simulation. The code also uses

```
print(...,"-dpng","-r300")
```

instead of simpler export commands, which improves the reliability and quality of generated PNG files.

Another optimization is the separation between plotting cadence and simulation cadence. The parameter `plotEvery` determines how often live visualization is updated, while `savePngEvery` controls how often PNG frames are exported. The numerical time step itself is unaffected by these values. This prevents expensive visualization from being performed at every step unless explicitly requested. In practice, the simulation may advance through many internal Euler steps while plotting only every twentieth step and exporting only every fiftieth step. Such decoupling is important because visualization frequency should be a user-controlled output setting rather than an implicit part of the solver algorithm.

The non-GUI runner `run_model_class.m` also includes reproducibility-oriented output management. For each run, it creates a timestamped output directory, stores a metadata file containing the parameters, MATLAB version, and platform information, writes a human-readable log file, and saves the final state to a MAT file. These measures do not alter the numerical solution, but they make the results traceable and easier to reproduce later. In a project where solver variants are benchmarked and compared, this output discipline is an important implementation choice.

The code contains additional robustness features intended to catch problems early. Input validation is used in several helper functions to verify that required fields are present and that dimensions are consistent. The step routine returns diagnostic information such as extrema and a flag for `NaN` or `Inf` values. The non-GUI runner checks this flag inside the main loop and stops immediately if non-finite values are detected. Such safeguards do not improve the mathematical order of the method, but they do improve the reliability of the software during testing and parameter exploration.

The project architecture also benefits from having multiple independent ways to exercise the same numerical core. The tests in the `tests` directory check Laplacian properties, stencil–matrix equivalence, timestep behavior, spatial convergence, mixed boundary conditions, manufactured-solution cases, Pearson-style pattern sanity, and GUI/non-GUI consistency. Because the solver components are modular, these tests can target specific layers of the program rather than only the complete application as a black box. This is one of the strengths of the implementation: correctness is supported not only by the final visual patterns, but also by structured verification of individual numerical and software components.

Although the GUI is not the focus of this chapter, it is worth noting its architectural role briefly. The App Designer file `GrayScottController.mlapp` is primarily a control and visualization layer. It allows the user to edit parameters, select presets, start or stop runs, reset the model, switch boundary types, and choose the diffusion backend. However, it does not replace the underlying numerical engine. Instead, it passes user choices into the same parameter and model infrastructure used by the non-GUI path. This keeps the implementation coherent: the interactive front end and the scripted solver remain two ways of using one common simulation core.

From a software-engineering perspective, the final implementation combines modularity, reproducibility, and optimization. The code avoids unnecessary rebuilding of operators, reuses graphics objects where repeated output is needed, isolates visualization from the solver, stores metadata for later comparison, and keeps backend-specific logic confined to a limited part of the code base. None of these decisions changes the mathematical scheme, but together they make the final solver more usable, easier to verify, and more suitable for systematic comparison.

In summary, the MATLAB implementation is organized around a clear division of responsibilities: parameter definition and normalization, grid construction, operator setup,

model-state management, one-step evolution, output handling, and testing are all separated into dedicated components. On top of this modular architecture, the final code introduces several practical optimizations, including preconstructed operators, reproducible initialization, boundary caching, persistent plotting objects, persistent export figures, and structured output logging. These implementation choices complement the numerical method and help turn the solver into a consistent and reliable computational tool.

4.7. Graphical User Interface (GUI)

4.7.1. GUI Design Evolution and Final Concept

The graphical user interface (GUI) of the final program was not fixed from the beginning, but was developed through a small design evolution. The main goal of the GUI was to provide a simple and practical interface for controlling the Gray–Scott simulation without changing the underlying numerical core. During the design process, different window layouts were considered in order to find a solution that was both user-friendly and technically robust in MATLAB.

Although the GUI is not the numerical core of the project, its implementation was not a trivial add-on. Even for a medium-sized scientific application, GUI development requires the coordination of many interactive elements such as parameter controls, boundary-condition selection, plotting, simulation-state handling, and recording options. For this reason, the GUI design had to be refined step by step instead of being treated as a purely cosmetic wrapper around the solver.

The first concept was an all-in-one interface in which the simulation controls and the graphical visualization were planned to appear in a single window. This idea was attractive because it grouped all user interactions and outputs in one place. Figure 4.2 shows this early mockup. However, during implementation this approach became less suitable, mainly because plotting inside an embedded axis object caused practical difficulties. In addition, combining all controls and visualization elements in one window made the interface more crowded and harder to manage.

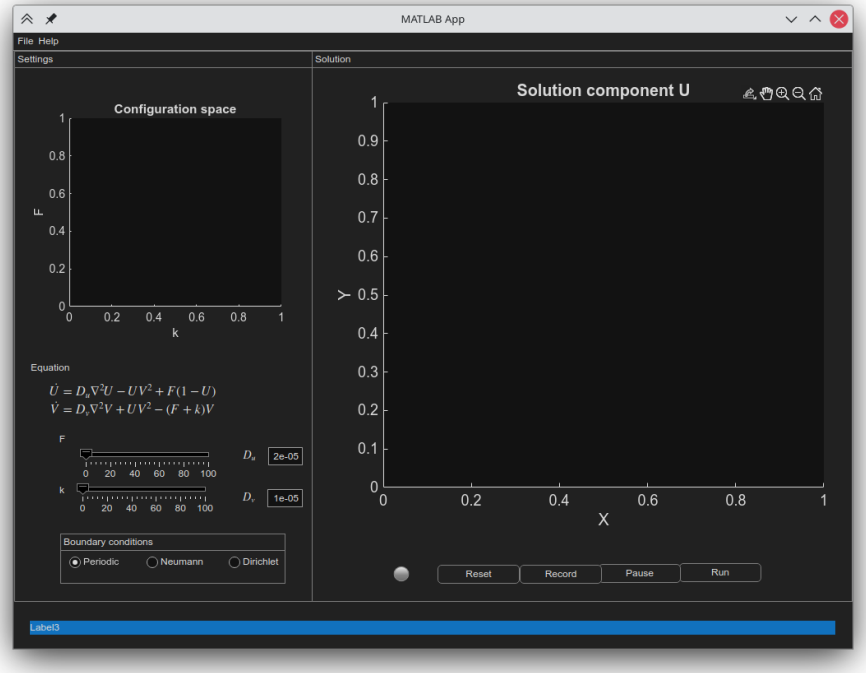

Figure 4.2.: Early all-in-one GUI mockup combining controls and simulation visualization in a single window.

For this reason, the design was revised toward a two-window concept. In this second approach, one window was dedicated to simulation control and parameter adjustment, while a separate window was used for visualization of the computed field. An early mockup of this direction is shown in Figure 4.3. This change reduced clutter, improved usability, and made the MATLAB implementation more manageable.

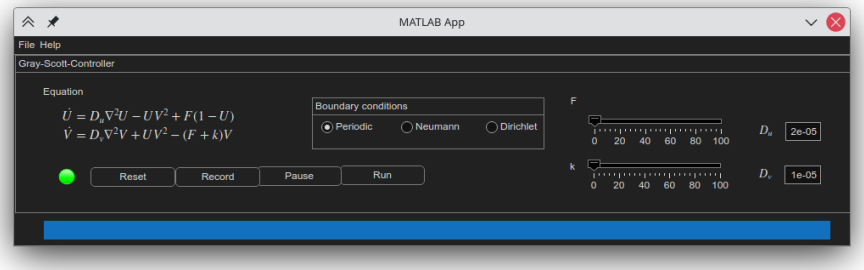

Figure 4.3.: Revised GUI mockup with separated controller and visualization windows.

The final GUI follows this separated architecture. A controller window is used for parameter input and simulation commands, while a separate plot window displays the selected solution field. This final concept provides a clearer workflow for the user and avoids the technical limitations encountered in the earlier all-in-one design. At the same time, it preserves the main purpose of the GUI as a convenient front end to the same simulation engine that is also used by the non-GUI workflow.

4.7.2. Final GUI Structure and Main Features

The final GUI is organized as two cooperating windows: a controller window for user interaction and a separate plot window for graphical output. This separation was chosen deliberately in order to keep the interface clear and to avoid overloading a single window with too many responsibilities. The controller is responsible for receiving user input and managing the simulation state, whereas the plot window is dedicated to visualizing the computed solution field.

The controller window is shown in Figure 4.4. It combines the most important simulation controls in one place. The interface includes parameter sliders for the feed and kill rates, editable diffusion coefficients, boundary-condition selection, and the main execution buttons for running, pausing, restarting, resetting, and recording the simulation. In addition, the user can choose which field is displayed, select a video format for export, define the recording frame rate, and monitor the current simulation state through the status display. The equations are also shown directly in the interface so that the GUI remains closely connected to the underlying mathematical model.

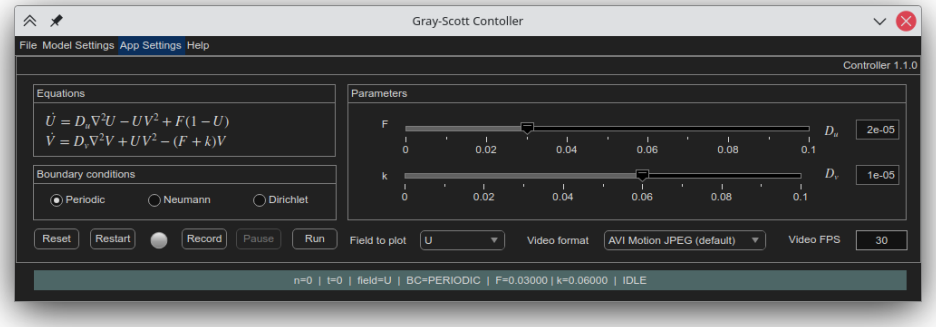

Figure 4.4.: Final controller window of the Gray–Scott GUI.

The visualization itself is handled in a separate plot window, shown in Figure 4.5. This window focuses on displaying the selected simulation field in a clean and readable form. By separating plotting from parameter control, the graphical output remains less cluttered

and easier to inspect during interactive runs. This also improves the practical usability of the program, since the user can focus on simulation control in one window and on pattern development in another.

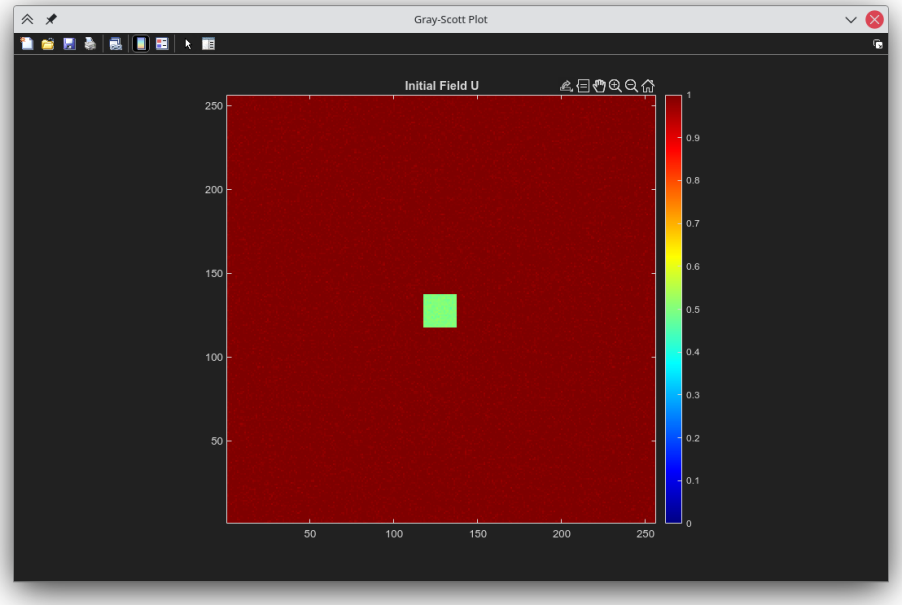

Figure 4.5.: Separate plot window used for visualization of the selected solution field.

Even at a high level, the GUI is more than a simple collection of buttons and sliders. Different user actions must trigger different updates in a coordinated way. Menu actions, parameter changes, plotting options, execution commands, boundary-condition selection, and recording settings all need to interact consistently with the current model state. The overall interaction logic is illustrated by the information-flow diagram in Figure 4.6. This figure emphasizes that building a GUI for a scientific simulation involves structured communication between many interface elements rather than only arranging visual components on the screen.

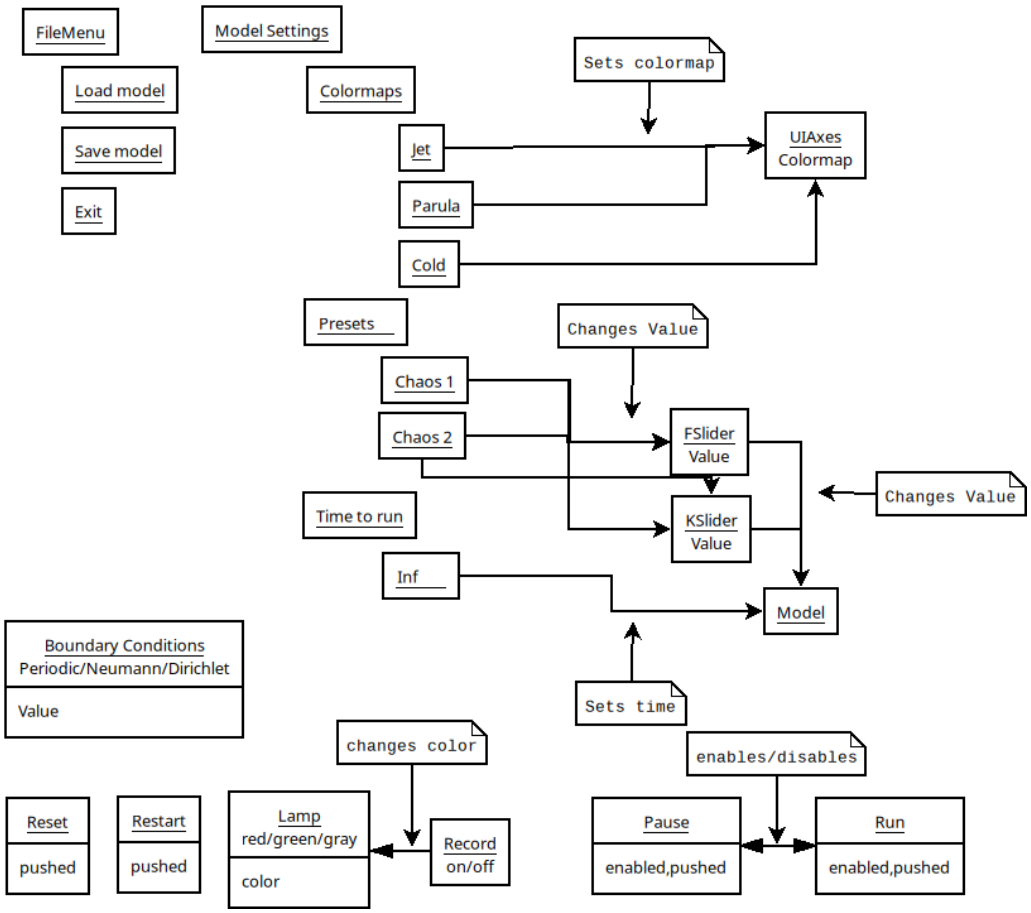

Figure 4.6.: High-level information flow between major GUI elements and user actions.

In this way, the final GUI provides an interface that is practical for interactive use while remaining closely connected to the numerical model. The chosen architecture makes the application easier to use, easier to maintain, and better suited to the workflow of repeated simulation, visualization, and optional recording.

4.7.3. Integration with the Simulation Core

The GUI was designed as a front end to the same simulation core that is also used outside the graphical environment. In other words, the GUI does not implement a separate numerical model of its own. Instead, it provides a structured way for the user to define parameters, start or stop computations, select visualization options, and trigger output actions while relying on the existing solver and model workflow in the background. This design was important in order to avoid duplication of numerical logic and to keep the program behavior consistent between GUI and non-GUI execution.

From a software point of view, the controller window acts as an interface layer between the user and the computational engine. User actions such as changing F, k, the diffusion coefficients, or the boundary condition do not directly create a new numerical method. Rather, they update the current simulation settings that are then passed to the same underlying model components used elsewhere in the project. The GUI therefore serves mainly as an organized control mechanism for the numerical implementation described in the previous sections of this chapter.

This separation between interface logic and numerical logic has several advantages. First, it improves consistency, because the same underlying implementation is used regardless of whether the simulation is launched interactively or by script. Second, it simplifies maintenance, since numerical changes do not need to be reimplemented separately inside the GUI. Third, it supports verification, because GUI-based execution can be compared against non-GUI execution under the same parameter settings.

A further practical benefit is that the GUI can focus on tasks that are specific to user interaction. These include enabling or disabling controls depending on the current simulation state, managing run/pause/restart behavior, updating status information, and coordinating visualization and recording options. The numerical solver itself remains part of the common simulation engine, while the GUI organizes how the user accesses that engine.

Overall, the final design follows a clear principle: the scientific computation is performed by the shared Gray–Scott model implementation, and the GUI provides a convenient interactive layer on top of it. This makes the program easier to use without weakening the structural connection between the interface and the verified numerical core.

5. Code Used in Original/Adopted Form

- **MATLAB:** MATLAB was the main software environment used to implement, execute, test, and benchmark the Gray–Scott simulation code. All numerical routines, verification scripts, benchmark programs, and export utilities of this project were developed and run in MATLAB.
- **MATLAB App Designer:** App Designer was used to build the graphical user interface of the project. It provides interactive control of model parameters, simulation start and restart actions, plotting options, and access to export functions.
- **VideoWriter:** MATLAB's built-in `VideoWriter` interface was used to export time-dependent simulation results as video files. This was used for recording the temporal evolution of patterns generated by the Gray–Scott model.
- **LaTeX:** LaTeX was used to write, structure, and format the final PPP report. It was used for the complete document preparation, including equations, figures, tables, and references.
- **TUBAF LaTeX template:** The official TUBAF report template was used as the formatting basis of the written report. It provided the required layout and document structure for the submission.
- **Biber/BibTeX bibliography workflow:** A bibliography toolchain was used to manage and format the references cited in the report. This made it possible to organize the literature sources consistently and include them automatically in the final document.

6. Usage of Chatbots

The integration of chatbots powered by advanced language models has become increasingly useful in project-based work. Such tools can provide rapid, context-aware assistance in technical and writing-related tasks, help identify possible issues, and support the preparation of structured documentation. Their ability to respond to natural-language queries and generate relevant suggestions can improve efficiency during development and reporting, provided that their output is reviewed critically and not used without verification.

In the context of this project, chatbot-based and AI-assisted tools were used in a limited and supportive role. In particular, ChatGPT and Gemini were used for several tasks during the development of the code and the preparation of the report. Grammarly was also used as an auxiliary language-support tool during the writing process.

The main areas of use were the following:

- **Code Review:** Chatbot support was used to review parts of the MATLAB code, identify possible weaknesses, and suggest improvements in structure, clarity, or consistency.
- **Debugging Support:** The tools were used to discuss possible causes of errors, unexpected behavior, or implementation problems and to generate suggestions for resolving them.
- **Code Comments:** Assistance was used in formulating or improving code comments so that the implementation became easier to read and understand.
- **Report Writing in LaTeX:** Chatbot support was used to help formulate technical text and convert report content into well-structured LaTeX code during the preparation of the written report.
- **Language Support:** Grammarly was used in a limited way to support wording, grammar, and language refinement in the final report.

The use of these tools was restricted to supportive assistance only. No chatbot-generated code was adopted directly without modification. Instead, suggestions were reviewed, adapted, and rewritten before being incorporated into the project. Likewise, no generated report text was inserted unchanged into the final document; all written content was manually revised and edited by the author. All parts of the project, including the implementation, results, code comments, and report text, were manually checked by the author before inclusion in the final submission.

Overall, the use of chatbots and AI-assisted language tools improved the efficiency of selected parts of the workflow, especially in code review, debugging discussion, commenting, and report preparation. However, the final technical decisions, implementation, verification, interpretation of results, and wording of the report remained entirely the responsibility of the author.

7. Program Flowchart

The execution structure of the final program is summarized in Figures 7.1 and 7.2. The first flowchart shows the initialization phase, and the second shows the repeated simulation loop.

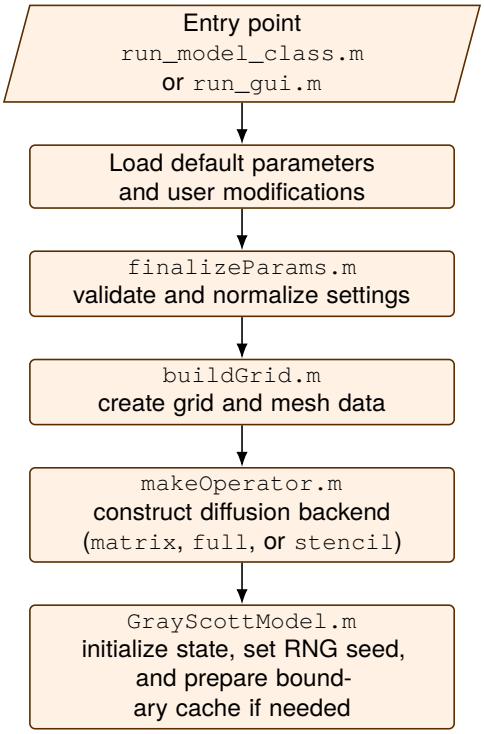

Figure 7.1.: Initialization and setup phase of the final Gray–Scott solver.

The setup phase shown in Figure 7.1 highlights that the final program follows a structured initialization sequence before time stepping begins. The execution starts either from the non-GUI runner `run_model_class.m` or from the graphical interface via `run_gui.m`. Default settings and user-defined modifications are then combined and passed through `finalizeParams.m`, where parameters are checked and normalized into a consistent form. After that, `buildGrid.m` creates the computational grid and mesh-related data, and `makeOperator.m` constructs the selected diffusion backend. Finally, the `GrayScottModel` object is created, the initial state is prepared, and all information required for the simulation loop is made available in a unified model representation.

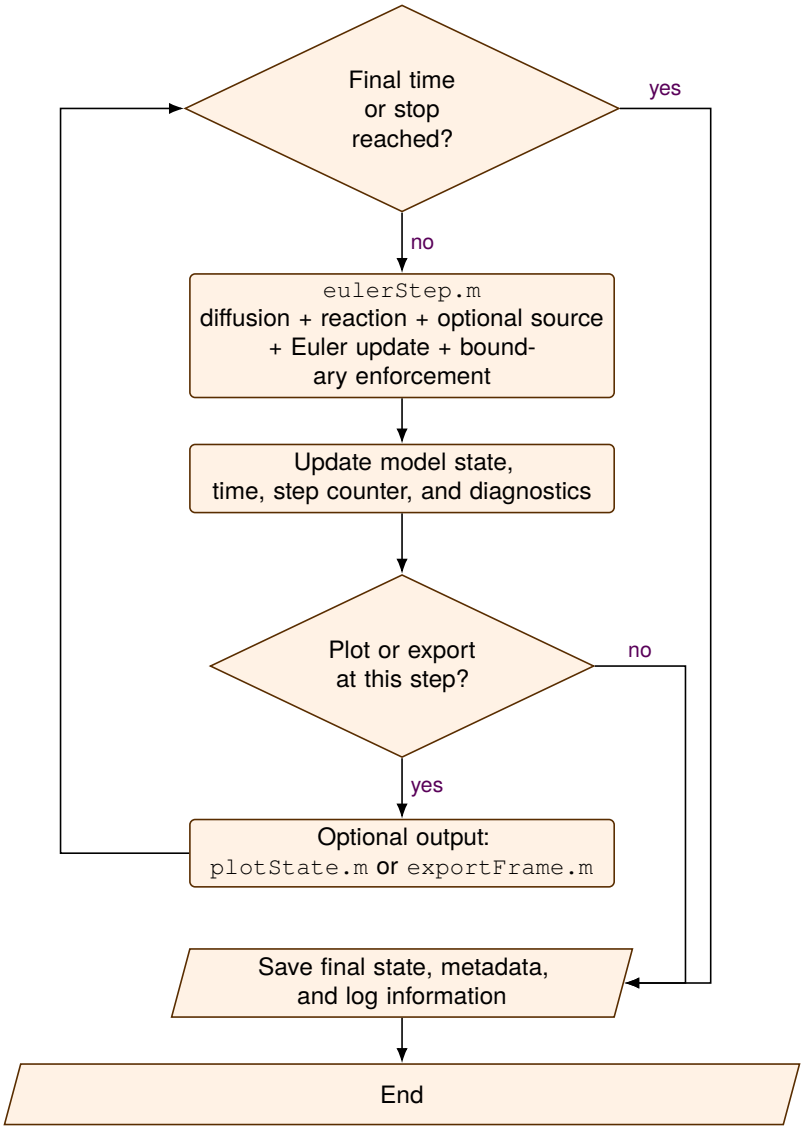

Figure 7.2.: Repeated simulation loop of the final Gray–Scott solver.

The loop phase in Figure 7.2 shows how the initialized model is advanced until the stopping criterion is reached. At each iteration, `eulerStep.m` performs the main numerical update by combining diffusion, reaction, optional source terms, and boundary enforcement in one time-step operation. The model state, current time, step counter, and diagnostic information are then updated consistently. Depending on the user settings, selected steps may trigger plotting or frame export, but these optional actions do not change the solver logic itself. Once the final time or stop condition is reached, the program stores the final state together with metadata and logging information before terminating in an orderly way.

8. Testing for Verification

8.1. Verification Strategy

This chapter documents the test cases used to verify the correctness and consistency of the implemented Gray–Scott solver. The focus is on *verification*, i.e. checking whether the numerical methods and their implementation solve the intended discrete problem correctly. In accordance with the PPP guideline, the chapter therefore emphasizes reproducible test cases with clearly defined aims, setups, execution procedures, expected results, and obtained results.

The verification strategy is organized in several layers. First, low-level operator tests are used to verify the correctness and convergence of the discrete diffusion operator. Second, time-integration tests check whether the explicit Euler method shows the expected temporal behavior. Third, boundary-condition tests confirm that Dirichlet and Neumann boundary conditions are imposed correctly in the implemented formulations. Fourth, equivalence tests compare different implementation variants, especially the matrix-based and stencil-based diffusion operators, in order to verify that they produce consistent results. Finally, interface-level consistency tests are used to check that the GUI execution path reproduces the same numerical results as the non-GUI reference workflow.

Besides these formal verification tests, the project also includes a validation-oriented pattern sanity check based on a Pearson-type Gray–Scott setup. This test is not a strict proof of numerical correctness in the sense of manufactured-solution verification, but it provides an additional qualitative sanity check that the implemented model reproduces physically plausible pattern-forming behavior. Since validation is not mandatory in the PPP guideline, such tests are treated here as supportive evidence rather than as the main verification basis.

Overall, the chapter is designed to move from rigorous numerical verification of core building blocks toward consistency checks of the full implementation. This layered structure is important because correctness of the final simulation results depends not only on the governing equations, but also on the correct interaction of operator assembly, time stepping, boundary treatment, solver implementation, and user interface execution.

8.2. Detailed Verification Test Cases

8.2.1. Test Case 1: MMS Operator Convergence of the Discrete Laplacian

Aim. This test case verifies the spatial accuracy of the discrete Laplacian operator implemented in the code. The purpose is to check that the matrix-based finite-difference Laplacian produced by `buildLaplacian2D` converges with the expected second-order accuracy on a sequence of refined grids when it is applied to a smooth exact solution. Since the Laplacian operator is one of the core building blocks of the Gray–Scott solver, this is a fundamental verification step.

Setup. The test uses the Method of Manufactured Solutions (MMS). A smooth periodic exact solution is prescribed on the square domain

$$\Omega = [0, 1) \times [0, 1)$$

with periodic boundary conditions in both spatial directions. The manufactured fields are

$$u^*(x, y, t) = 1.0 + 0.1 \cos\big(2\pi(2x + 3y) + t\big),$$
$$v^*(x, y, t) = 0.5 + 0.1 \cos\big(2\pi(2x + 3y) + t\big).$$

The corresponding exact Laplacians are known analytically:

$$\nabla^2 u^*(x, y, t) = -0.1(2\pi)^2(2^2 + 3^2) \cos\big(2\pi(2x + 3y) + t\big),$$
$$\nabla^2 v^*(x, y, t) = -0.1(2\pi)^2(2^2 + 3^2) \cos\big(2\pi(2x + 3y) + t\big).$$

The test evaluates the operator at the fixed time

$$t_0 = 0.3.$$

Uniform grids with

$$N_x = N_y \in \{32, 64, 128, 256\}$$

are used. Since $L_x = L_y = 1$, the grid spacing is

$$h = \frac{1}{N}.$$

For each grid, the sparse discrete Laplacian matrix L is assembled and applied to the exact grid values of u^* and v^*. The numerical result is compared against the analytic Laplacian, and the discrete L^2 error is computed.

How to run the test case. From the project `tests` folder, run:

```
r = test_mms_operator_convergence();
```

The test prints the error values in the MATLAB command window and saves the convergence plot to:

```
figures/verification/mms_operator_convergence.png
```

Expected result. For a smooth periodic solution on a uniform grid, the standard five-point finite-difference Laplacian is expected to be second-order accurate in space. Therefore, the discrete L^2 error should satisfy

$$\|Lu^* - \nabla^2 u^*\|_{L^2} = \mathcal{O}(h^2), \qquad \|Lv^* - \nabla^2 v^*\|_{L^2} = \mathcal{O}(h^2).$$

As a consequence, halving the grid spacing should reduce the error by approximately a factor of four. On a log–log plot of error versus h, the measured slope should be close to 2.

Obtained result. The test produced the error values listed in Table 8.1. The corresponding convergence plot is shown in Figure 8.1.

Table 8.1.: Measured L^2 errors for the discrete Laplacian MMS operator test.

$N_x = N_y$	h	$\|Lu^* - \nabla^2 u^*\|_{L^2}$	$\|Lv^* - \nabla^2 v^*\|_{L^2}$
32	0.03125	8.609×10^{-1}	8.609×10^{-1}
64	0.015625	2.169×10^{-1}	2.169×10^{-1}
128	0.0078125	5.434×10^{-2}	5.434×10^{-2}
256	0.00390625	1.359×10^{-2}	1.359×10^{-2}

The observed convergence orders reported by the code are

$$p_u = 2.00, \qquad p_v = 2.00.$$

This exactly matches the theoretical expectation for a second-order finite-difference Laplacian. In addition, the error decreases by approximately a factor of four whenever the grid is refined by a factor of two:

$$0.8609 \to 0.2169 \to 0.05434 \to 0.01359.$$

This behavior is fully consistent with $\mathcal{O}(h^2)$ convergence.

The u- and v-errors are identical in this test because both manufactured fields use the same oscillatory spatial structure and differ only in their constant offsets, whose Laplacians are zero. Therefore, the test confirms that the implemented discrete Laplacian is correct and achieves the expected second-order spatial accuracy.

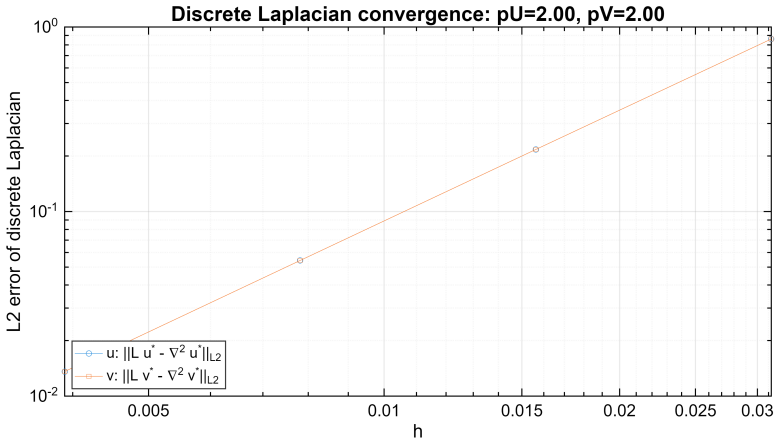

Figure 8.1.: Log–log convergence plot for the discrete Laplacian MMS test.

8.2.2. Test Case 2: Temporal Convergence of the Explicit Euler Scheme

Aim. This test case verifies the temporal behavior of the explicit Euler time-integration scheme used in the Gray–Scott solver. Since explicit Euler is theoretically first-order accurate in time, the purpose of the test is to check whether the numerical solution shows the expected first-order convergence under time-step refinement while all spatial discretization settings are kept fixed.

Setup. The test is performed for the baseline Gray–Scott configuration on the square domain

$$\Omega = [0, 1) \times [0, 1),$$

using a uniform grid with

$$N_x = N_y = 128.$$

The model parameters are taken from `defaultParams()`:

$$D_u = 2 \times 10^{-5}, \qquad D_v = 1 \times 10^{-5}, \qquad F = 0.03, \qquad k = 0.06.$$

Periodic boundary conditions are used on all sides. The final simulation time is

$$T = 50.$$

The initial condition is the standard baseline initialization used in the project: the field U is initialized close to 1, the field V close to 0, and a centered 10×10 square patch is imposed with

$$U = 0.50, \qquad V = 0.25,$$

followed by a small random perturbation with amplitude $\texttt{icPerturb} = 0.02$. In order to make the comparison reproducible, the random number generator is reset with the same seed before all runs, so that each simulation starts from exactly the same initial state.

The same problem is then solved three times with the matrix-based diffusion operator and with three different time-step sizes:

$$\Delta t \in \{0.2,\ 0.1,\ 0.05\}.$$

At the final time T, the v-field of the three runs is compared. Since no exact time-dependent reference solution is used in this test, a self-convergence approach is applied through the successive differences

$$d_{12} = \|v_{\Delta t} - v_{\Delta t/2}\|, \qquad d_{24} = \|v_{\Delta t/2} - v_{\Delta t/4}\|.$$

These differences are evaluated in both the discrete L^2-norm and the discrete L^∞-norm.

How to run the test case. From the project `tests` folder, run:

```
r = test_timestep_convergence();
```

The test prints the measured successive differences and observed orders to the MATLAB command window and saves the figure

```
figures/verification/timestep_convergence.png
```

Expected result. For a first-order time-integration method, the temporal discretization error behaves as

$$e(\Delta t) = C\,\Delta t + \mathcal{O}(\Delta t^2).$$

Therefore, in the asymptotic regime, the successive-difference indicators are also expected to scale linearly with Δt. In particular, halving the time step should approximately halve the difference, so that

$$\frac{d_{12}}{d_{24}} \approx 2.$$

The observed order estimate

$$p = \frac{\log(d_{12}/d_{24})}{\log 2}$$

should therefore be close to 1. In addition, the finer-step difference d_{24} should be smaller than the coarser-step difference d_{12}.

Obtained result. The measured results are listed in Table 8.2, and the L^2-based refinement plot is shown in Figure 8.2. The numerical results are

$$d_{12}^{L^2} = 3.706 \times 10^{-5}, \qquad d_{24}^{L^2} = 1.851 \times 10^{-5}, \qquad p_{L^2} = 1.001,$$

and

$$d_{12}^{L^\infty} = 5.798 \times 10^{-4}, \qquad d_{24}^{L^\infty} = 2.891 \times 10^{-4}, \qquad p_{L^\infty} = 1.004.$$

Table 8.2.: Measured successive differences and observed orders for the explicit Euler time-step refinement test.

Norm	d_{12}	d_{24}	Observed order p
L^2	3.706×10^{-5}	1.851×10^{-5}	1.001
L^∞	5.798×10^{-4}	2.891×10^{-4}	1.004

The finer-step difference is smaller than the coarser-step difference in both norms, as expected:

$$d_{24}^{L^2} < d_{12}^{L^2}, \qquad d_{24}^{L^\infty} < d_{12}^{L^\infty}.$$

Moreover, the ratio between successive differences is essentially 2, which yields an observed order of approximately 1 in both norms. This is in excellent agreement with the theoretical first-order accuracy of the explicit Euler method.

Therefore, the test confirms that the implemented time-integration scheme behaves correctly under time-step refinement and that the solver is operating in the expected temporal convergence regime for this problem configuration.

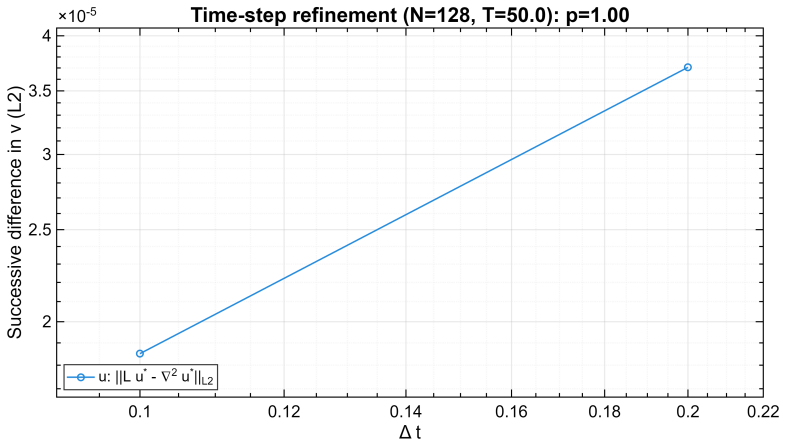

Figure 8.2.: Time-step refinement test for the explicit Euler scheme. The plot shows the L^2-based successive differences of the v-field for $\Delta t = 0.2$ and $\Delta t = 0.1$, indicating first-order temporal convergence.

8.2.3. Test Case 3: Verification of Boundary Condition Enforcement

Aim. This test case verifies that the implemented boundary-condition treatment is applied correctly and robustly in the Gray–Scott solver. Since support for non-periodic boundaries is an explicit feature of the project, it is important to check not only that Dirichlet and Neumann conditions are imposed correctly at the boundary nodes, but also that this enforcement remains stable during time integration and is consistent across all implemented diffusion modes.

Setup. The test is performed on the square domain

$$\Omega = [0, 1) \times [0, 1)$$

using a uniform grid with

$$N_x = N_y = 64.$$

The time step is

$$\Delta t = 0.25,$$

and the test is repeated for all three diffusion implementations:

$$\texttt{matrix}, \quad \texttt{stencil}, \quad \texttt{full}.$$

The verification script combines three sub-checks within one test case.

(1) Dirichlet constant enforcement. A smooth nontrivial initial field is prescribed for both species,

$$U_0(x, y) = 0.80 + 0.10 \sin(2\pi x) \cos(2\pi y),$$
$$V_0(x, y) = 0.20 + 0.05 \cos(2\pi x) \sin(2\pi y),$$

and a constant Dirichlet condition is imposed on the left boundary:

$$u = 0.123, \qquad v = 0.456.$$

The other three boundaries remain periodic. The simulation is advanced for 20 time steps, and after each step the maximum deviation of the left boundary values from the prescribed constants is recorded.

(2) Neumann derivative consistency. The same smooth initial fields are used again, but now a constant Neumann condition is prescribed on the left boundary:

$$\frac{\partial u}{\partial n} = 1.0, \qquad \frac{\partial v}{\partial n} = -2.0.$$

The other boundaries remain periodic. The run is again advanced for 20 time steps. After each step, the normal derivative at the left boundary is approximated by the discrete one-sided difference

$$\frac{U(:,2)-U(:,1)}{h_x}, \qquad \frac{V(:,2)-V(:,1)}{h_x},$$

and the maximum deviation from the prescribed derivative values is recorded.

(3) Mixed boundary-condition robustness test. A more demanding long-time test is then performed with a strongly mismatched initial state,

$$U_0 = 0.95, \qquad V_0 = 0.05$$

throughout the interior, while the following boundary conditions are imposed:

$$u = 0.2, \quad v = 0.4 \qquad \text{on the left boundary (Dirichlet)},$$

$$\frac{\partial u}{\partial n} = 0, \quad \frac{\partial v}{\partial n} = 0 \qquad \text{on the right boundary (Neumann)},$$

with periodic boundaries at the top and bottom. This case is run for 400 time steps, corresponding to

$$T = 100.$$

During the run, the maximum boundary errors are tracked for both the Dirichlet and Neumann sides. In addition, an interior monitoring column is monitored in order to confirm that the solution evolves nontrivially in the interior and that the test is not merely a static boundary overwrite. The script also checks that no `NaN` or `Inf` values occur.

How to run the test case. From the project `tests` folder, run:

```
r = test_boundary_conditions();
```

The test prints the maximum boundary errors for all three diffusion modes in the MATLAB command window.

Expected result. For the Dirichlet sub-test, the prescribed boundary values should be enforced exactly at every time step, up to machine precision. Therefore, the maximum boundary error should be essentially zero.

For the Neumann sub-test, the discrete boundary gradient should match the prescribed normal derivative. Since the condition is imposed directly in the discrete formulation, the resulting error should again be at the level of roundoff.

For the mixed long-time test, the left Dirichlet boundary should remain fixed at its prescribed values and the right Neumann boundary should remain zero-flux throughout the simulation. At the same time, the interior should evolve measurably away from the initial state. Thus, the expected result is:

- boundary errors at or near machine precision,
- no `NaN` or `Inf` values,
- nonzero interior change change,
- and identical qualitative behavior for `matrix`, `stencil`, and `full` modes.

Obtained result. All checks passed for all three diffusion modes. The returned test result was

$$\texttt{pass} = 1,$$

and the script reported that the quantitative Dirichlet, Neumann, and mixed boundary-condition checks passed for `matrix`, `stencil`, and `full` modes.

The measured results are summarized separately in Tables 8.3, 8.4, and 8.5. For the Dirichlet test, the maximum boundary error was exactly zero for both u and v in all three modes over 20 steps. For the Neumann test, the maximum error was zero for u and

$$3.553 \times 10^{-15}$$

for v, which is at the level of machine precision and therefore consistent with exact discrete enforcement up to floating-point roundoff.

In the mixed 400-step test, the maximum Dirichlet and Neumann boundary errors remained zero for both species in all three diffusion modes. At the same time, the interior change values changed by

$$2.992 \times 10^{-2} \quad \text{for } u, \qquad 4.982 \times 10^{-2} \quad \text{for } v,$$

which confirms that the solution evolved nontrivially in the interior while the imposed boundary conditions remained exactly enforced.

The fact that the reported errors are identical for `matrix`, `stencil`, and `full` is a strong indication that the boundary-condition handling is implemented consistently across all diffusion formulations.

Table 8.3.: Dirichlet boundary-condition verification results for all diffusion modes.

Mode	maxErr u	maxErr v
`matrix`	0.000×10^{0}	0.000×10^{0}
`stencil`	0.000×10^{0}	0.000×10^{0}
`full`	0.000×10^{0}	0.000×10^{0}

Table 8.4.: Neumann boundary-condition verification results for all diffusion modes.

Mode	maxErr u	maxErr v
`matrix`	0.000×10^{0}	3.553×10^{-15}
`stencil`	0.000×10^{0}	3.553×10^{-15}
`full`	0.000×10^{0}	3.553×10^{-15}

Table 8.5.: Mixed boundary-condition robustness results for all diffusion modes after 400 steps.

Mode	maxDir u	maxDir v	maxNeu u	maxNeu v	interior change Δu	interior change Δv
`matrix`	0.000×10^{0}	0.000×10^{0}	0.000×10^{0}	0.000×10^{0}	2.992×10^{-2}	4.982×10^{-2}
`stencil`	0.000×10^{0}	0.000×10^{0}	0.000×10^{0}	0.000×10^{0}	2.992×10^{-2}	4.982×10^{-2}
`full`	0.000×10^{0}	0.000×10^{0}	0.000×10^{0}	0.000×10^{0}	2.992×10^{-2}	4.982×10^{-2}

Overall, this test case confirms that the boundary-condition treatment is not only mathematically correct at the discrete level, but also numerically robust during time stepping and consistent across all solver variants used in the project.

8.2.4. Test Case 4: Matrix–Stencil Equivalence of the Diffusion Operator

Aim. This test case verifies that the two implemented diffusion-operator realizations, namely the sparse matrix formulation and the matrix-free stencil formulation, produce the same numerical result when applied to the same discrete field. Since both implementations are intended to represent the same five-point finite-difference Laplacian on a periodic grid, this test is an important consistency check at the operator level.

Setup. The test compares the output of the sparse matrix Laplacian assembled by `buildLaplacian2D` with the output of the matrix-free stencil operator evaluated by `applyLaplacianStencil`. The comparison is carried out for periodic connectivity only, because physical boundary conditions such as Dirichlet and Neumann are imposed separately after each time step and are therefore not part of this operator-equivalence test.

Three different grid sizes are used in order to detect possible indexing, reshaping, or dimension-ordering mistakes:

$$32 \times 32, \qquad 64 \times 48, \qquad 128 \times 128.$$

For each grid, three independent random trials are performed. In each trial, a random discrete field u is generated, and the following two operator evaluations are computed:

$$a = Lu, \qquad b = \mathcal{S}(u),$$

where L denotes the sparse matrix Laplacian and $\mathcal{S}$ denotes the stencil-based Laplacian application.

The difference between the two results is measured using:

$$\text{relative error} = \frac{\|a - b\|_2}{\max(1, \|a\|_2)},$$

and

$$\text{maximum absolute difference} = \max_i |a_i - b_i|.$$

The tolerances used by the test are

$$\texttt{relTol} = 10^{-12}, \qquad \texttt{absTol} = 10^{-10}.$$

How to run the test case. From the project `tests` folder, run:

```
r = test_laplacian_stencil_equivalence();
```

The test prints the relative and absolute differences for each grid and trial and reports the worst observed case.

Expected result. Both implementations represent the same discrete Laplacian operator. Therefore, for the same input vector u, their outputs should agree up to floating-point roundoff. As a result, the relative error should be extremely small, and the maximum absolute difference should remain below the prescribed numerical tolerances. In particular, all trials should pass with margins far below

$$10^{-12} \quad \text{(relative)} \qquad \text{and} \qquad 10^{-10} \quad \text{(absolute)}.$$

Obtained result. All nine test cases passed successfully, and the returned result was

$$\texttt{pass} = 1.$$

The measured values are listed in Table 8.6. The relative errors were consistently on the order of 10^{-16}, which is close to machine precision. The largest observed relative error was

$$1.249 \times 10^{-16}$$

for the 64×48 grid in trial 2. The largest observed maximum absolute difference was

$$5.821 \times 10^{-11}$$

for the 128×128 grid in trial 1.

These worst-case values are still safely below the prescribed tolerances:

$$1.249 \times 10^{-16} \ll 10^{-12}, \qquad 5.821 \times 10^{-11} < 10^{-10}.$$

The slightly larger absolute differences on the finest grid are expected because larger systems may accumulate somewhat more floating-point roundoff, but the differences remain numerically negligible.

Therefore, the test confirms that the matrix-based and stencil-based diffusion operators are equivalent to machine precision and can be regarded as two consistent implementations of the same finite-difference Laplacian.

Table 8.6.: Matrix–stencil operator equivalence results for all tested grids and trials.

Grid	Trial	Relative error	Max absolute difference
32×32	1	1.145×10^{-16}	3.638×10^{-12}
32×32	2	1.127×10^{-16}	1.819×10^{-12}
32×32	3	1.132×10^{-16}	1.819×10^{-12}
64×48	1	1.211×10^{-16}	7.276×10^{-12}
64×48	2	1.249×10^{-16}	1.455×10^{-11}
64×48	3	1.181×10^{-16}	7.276×10^{-12}
128×128	1	1.131×10^{-16}	5.821×10^{-11}
128×128	2	1.132×10^{-16}	5.821×10^{-11}
128×128	3	1.118×10^{-16}	5.821×10^{-11}

8.2.5. Test Case 5: End-to-End Equivalence of Matrix and Stencil Solver Modes

Aim. This test case verifies that the matrix-based and stencil-based diffusion modes are equivalent at the level of the full Gray–Scott simulation, not only at the isolated operator level. In contrast to the previous operator-equivalence test, this case checks the complete solver workflow, including model initialization, time stepping, reaction evaluation, diffusion application, and state updates. Its purpose is to confirm that both solver modes produce the same final numerical solution when they are run with identical settings.

Setup. The test uses the full `GrayScottModel` class on the square periodic domain

$$\Omega = [0, 1) \times [0, 1),$$

with a uniform grid

$$N_x = N_y = 128.$$

The standard project parameters from `defaultParams()` are used:

$$D_u = 2 \times 10^{-5}, \qquad D_v = 1 \times 10^{-5}, \qquad F = 0.03, \qquad k = 0.06.$$

The time step is

$$\Delta t = 0.2,$$

and the final simulation time is

$$T = 5.$$

Two runs are then carried out:

- a baseline run with `diffusionMode = "matrix"`,
- a comparison run with `diffusionMode = "stencil"`.

To ensure that both runs start from exactly the same initial condition, the random number generator is reset to the same seed before each run. The same grid and the same model parameters are used in both cases. After both simulations reach the final time T, the final states are compared componentwise.

Let u_A, v_A denote the final matrix-mode solution and u_B, v_B the final stencil-mode solution. The test evaluates the differences

$$d_u = u_A - u_B, \qquad d_v = v_A - v_B,$$

and computes both relative and maximum absolute differences:

$$\mathrm{rel}_u = \frac{\|d_u\|_2}{\max(1, \|u_A\|_2)}, \qquad \mathrm{rel}_v = \frac{\|d_v\|_2}{\max(1, \|v_A\|_2)},$$

$$\max_u = \max_i |d_{u,i}|, \qquad \max_v = \max_i |d_{v,i}|.$$

The tolerances used by the test are

$$\texttt{relTol} = 10^{-10}, \qquad \texttt{absTol} = 10^{-9}.$$

How to run the test case. From the project `tests` folder, run:

```
r = test_matrix_stencil_endtoend_equivalence();
```

The test prints the measured relative and maximum absolute differences in the MATLAB command window.

Expected result. If the matrix-based and stencil-based diffusion modes are truly equivalent implementations of the same numerical method, then the complete Gray–Scott runs should produce the same final solution up to floating-point roundoff. Therefore, the relative differences in both u and v should be extremely small, and the maximum absolute differences should remain far below the prescribed tolerances. In particular, both solution components should satisfy

$$\mathrm{rel}_u < 10^{-10}, \qquad \mathrm{rel}_v < 10^{-10},$$

and

$$\mathrm{max}_u < 10^{-9}, \qquad \mathrm{max}_v < 10^{-9}.$$

Obtained result. The test passed successfully, and the returned result was

$$\mathtt{pass} = 1.$$

The measured final-state differences are summarized in Table 8.7.

Table 8.7.: End-to-end comparison between matrix and stencil solver modes at final time $T = 5$.

Quantity	Measured value
rel_u	8.903×10^{-17}
rel_v	9.927×10^{-17}
max_u	4.441×10^{-16}
max_v	5.551×10^{-17}

All reported differences are many orders of magnitude smaller than the specified tolerances:

$$8.903 \times 10^{-17} \ll 10^{-10}, \qquad 9.927 \times 10^{-17} \ll 10^{-10},$$
$$4.441 \times 10^{-16} \ll 10^{-9}, \qquad 5.551 \times 10^{-17} \ll 10^{-9}.$$

Thus, the final solutions produced by the two solver modes are numerically indistinguishable up to machine precision.

This result shows that the stencil-based and matrix-based implementations remain consistent not only when applying the Laplacian to a test vector, but also when embedded in the full Gray–Scott time integration process.

8.2.6. Test Case 6: GUI and Non-GUI Consistency

Aim. This test case verifies that the graphical user interface does not alter the numerical model and that a simulation executed through the GUI reproduces exactly the same numerical results as the corresponding non-GUI workflow. The purpose is to check

consistency at the interface level, i.e. to confirm that the GUI acts only as a control and visualization layer and does not introduce unintended changes in parameters, initialization, or solver behavior.

Setup. The consistency check is carried out in three stages. First, a non-GUI reference run is generated using the same parameter set as the GUI startup configuration. Second, the GUI is launched with its current shipped default setup and a dedicated helper script advances the GUI-owned model for the exact number of required time steps. Third, the saved non-GUI and GUI results are compared quantitatively.

For the run documented here, the GUI startup configuration was:

$$\texttt{preset} = \texttt{pearson}, \qquad \texttt{diffusionMode} = \texttt{stencil},$$

with periodic boundary conditions on all sides. The computational grid was

$$N_x = N_y = 256,$$

the time step was

$$\Delta t = 0.5,$$

and the final simulation time was

$$T = 500.$$

Therefore, the total number of time steps was

$$n_{\text{steps}} = \frac{T}{\Delta t} = \frac{500}{0.5} = 1000.$$

A midpoint snapshot was stored at

$$k_{\text{snap}} = 500.$$

The comparison includes both the midpoint snapshot and the final state for the two species u and v. For each compared field, the following quantities are evaluated:

$$\text{relL2} = \frac{\|w_{\text{GUI}} - w_{\text{ref}}\|_2}{\max(1, \|w_{\text{ref}}\|_2)}, \qquad \text{maxAbs} = \max_i |w_{\text{GUI},i} - w_{\text{ref},i}|.$$

The tolerances used by the comparison script are

$$\texttt{tol\_relL2} = 10^{-12}, \qquad \texttt{tol\_maxAbs} = 10^{-12}.$$

The relevant scripts are:

```
tests/gui_consistency/run_nongui_reference_dump.m
tests/gui_consistency/run_gui_consistency_dump.m
tests/gui_consistency/compare_gui_nongui.m
```

How to run the test case. Run the workflow either from the project root or from `tests/gui_consistency`. Make sure that the project paths are available. Then execute:

```
ref = run_nongui_reference_dump();
gui = run_gui_consistency_dump();
T   = compare_gui_nongui(ref.refFile, gui.guiFile);
```

For the documented run, the scripts saved:

```
results/consistency/nongui_reference_<timestamp>.mat
results/consistency/gui_dump_<timestamp>.mat
results/consistency/gui_consistency_table.csv
figures/verification/gui_consistency.png
```

Expected result. If the GUI is consistent with the underlying solver implementation, then the GUI-generated run and the non-GUI reference run should produce identical or numerically indistinguishable data when started from the same configuration. Since this test uses the same model parameters, same grid, same seed, same number of steps, and the same solver configuration, the expected result is that both the midpoint snapshot and the final state match within the very strict tolerances

$$10^{-12}$$

for both the relative L^2 difference and the maximum absolute difference.

Obtained result. The comparison passed in all four checked cases: snapshot u, snapshot v, final u, and final v. The measured results are summarized in Table 8.8. In every case,

$$\mathrm{relL2} = 0, \qquad \mathrm{maxAbs} = 0,$$

and the pass flag was `true`. Therefore, the GUI and non-GUI workflows produced exactly identical saved states for both the midpoint snapshot and the final solution.

Table 8.8.: GUI and non-GUI consistency comparison results.

Stage	Field	relL2	maxAbs	Pass
snapshot	u	0	0	true
snapshot	v	0	0	true
final	u	0	0	true
final	v	0	0	true

Figure 8.3 shows the GUI–non-GUI difference field for the snapshot of u. The plotted field is identically zero, which is fully consistent with the tabulated comparison metrics. The same conclusion applies to the other compared states as well.

Hence, this test confirms that the GUI layer does not modify the mathematical or numerical behavior of the solver. It reproduces the same results as the non-GUI reference workflow exactly for the tested configuration.

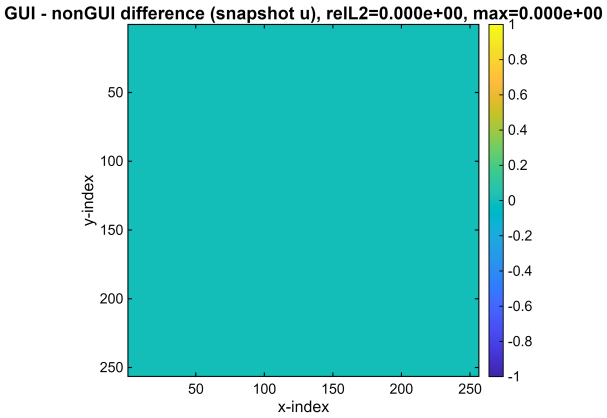

Figure 8.3.: Difference field between GUI and non-GUI results for the snapshot of u. The field is identically zero, confirming exact agreement for the documented run.

8.2.7. Test Case 7: Gray–Scott Grid Refinement Test

Aim. This test case examines how the fully nonlinear Gray–Scott solution changes under spatial grid refinement. In contrast to the manufactured-solution tests, this is not a strict exact-solution verification test, because no analytical reference solution is available for the nonlinear pattern-forming problem considered here. Instead, the purpose is to check whether the computed solution approaches a fine-grid reference in a consistent and systematic way as the spatial grid is refined. This provides additional evidence that the implemented solver behaves plausibly for the actual Gray–Scott system, beyond the linearized or manufactured-solution settings used in the earlier test cases.

Setup. The test is performed for the Gray–Scott model on the periodic square domain

$$\Omega = [0, 1) \times [0, 1)$$

using the standard baseline parameter set

$$D_u = 2 \times 10^{-5}, \qquad D_v = 1 \times 10^{-5}, \qquad F = 0.03, \qquad k = 0.06.$$

The final simulation time is

$$T = 1.$$

The problem is solved on the sequence of uniform grids

$$N_x = N_y \in \{64,\ 128,\ 256,\ 512,\ 1024\}.$$

The finest grid,

$$N_x = N_y = 1024,$$

is used as the numerical reference solution. For the coarser grids, the corresponding solution is compared against the finest-grid result after interpolation of the reference field onto the coarser mesh using cubic interpolation.

To reduce the influence of temporal discretization error on the refinement study, the time step is chosen proportional to

$$h^2.$$

More precisely, the test uses

$$\Delta t = 0.01 \quad \text{for } N = 64,$$

and then scales Δt by a factor of four whenever the grid spacing is halved. This gives:

$$\Delta t \in \left\{0.01,\ 0.0025,\ 0.000625,\ 0.00015625,\ 3.90625 \times 10^{-5}\right\}.$$

Accordingly, the corresponding numbers of time steps are

$$N_t \in \{100,\ 400,\ 1600,\ 6400,\ 25600\}.$$

For each coarser grid, the discrete L^2 differences between the computed solution and the interpolated finest-grid reference are measured for both species:

$$e_U = \|U_h - U_{\mathrm{ref}}\|_{L^2}, \qquad e_V = \|V_h - V_{\mathrm{ref}}\|_{L^2}.$$

Observed refinement rates p_U and p_V are then estimated from the log–log slope of error versus grid spacing.

How to run the test case. From the project `tests` folder, run:

```
r = test_grayscott_grid_refinement();
```

The test prints the completed grid levels, interpolates the reference solution onto the coarser grids, reports the measured L^2 differences, and saves the refinement plot to:

```
figures/verification/grayscott_grid_refinement.png
```

Expected result. Since the finest-grid solution is used as a numerical reference, one expects the coarse-grid errors to decrease monotonically as the spatial grid is refined. If the implementation behaves consistently, the solutions on successively finer grids should approach the fine-grid reference, and the corresponding L^2 differences should become smaller.

For this nonlinear pattern-forming problem, however, one should not expect a perfectly clean textbook second-order rate in the same sense as in the manufactured-solution operator test. Even when the numerical method itself is second-order in space, small phase shifts in the evolving pattern can produce visible pointwise differences between two solutions that are otherwise qualitatively very similar. Therefore, in this test the main criterion is consistent error reduction under refinement, together with a clearly positive observed refinement rate. The time step is scaled as $\Delta t \sim h^2$ specifically to suppress temporal error as much as possible, so that the measured differences are dominated mainly by spatial effects.

Obtained result. The test completed successfully, and the returned result was

$$\texttt{pass} = 1.$$

The MATLAB output reported the following completed runs:

$$N = 64\ (\Delta t = 0.01,\ N_t = 100), \quad N = 128\ (\Delta t = 0.0025,\ N_t = 400),$$

$$N = 256\ (\Delta t = 0.000625,\ N_t = 1600), \quad N = 512\ (\Delta t = 0.00015625,\ N_t = 6400),$$

$$N = 1024\ (\Delta t = 3.90625 \times 10^{-5},\ N_t = 25600).$$

The finest-grid reference was then interpolated onto the 64×64, 128×128, 256×256, and 512×512 grids using cubic interpolation.

The measured L^2 differences are listed in Table 8.9. In both U and V, the error decreases monotonically as the grid is refined:

$$7.964 \times 10^{-2} \to 3.747 \times 10^{-2} \to 1.571 \times 10^{-2} \to 5.010 \times 10^{-3}$$

for U, and

$$4.332 \times 10^{-2} \to 2.186 \times 10^{-2} \to 9.693 \times 10^{-3} \to 3.645 \times 10^{-3}$$

for V.

The observed refinement rates reported by the code are

$$p_U = 1.70, \qquad p_V = 1.63.$$

These values are clearly positive and indicate substantial convergence toward the finest-grid reference, but they are somewhat below the ideal value 2. For the nonlinear Gray–Scott problem, this is not unexpected. As the patterns evolve, small phase shifts and

Table 8.9.: Gray–Scott grid-refinement differences relative to the interpolated 1024×1024 reference solution.

$N_x = N_y$	h	$\|e_U\|_{L^2}$	$\|e_V\|_{L^2}$
64	0.015625	7.964×10^{-2}	4.332×10^{-2}
128	0.0078125	3.747×10^{-2}	2.186×10^{-2}
256	0.00390625	1.571×10^{-2}	9.693×10^{-3}
512	0.001953125	5.010×10^{-3}	3.645×10^{-3}

slight spatial displacements can develop between solutions computed on different grids. Such phase differences can increase the pointwise and L^2 discrepancy even when the overall morphology is converging correctly. Therefore, the measured rates should be interpreted as consistency-oriented refinement rates for a nonlinear pattern-forming PDE rather than as exact textbook spatial orders.

Figure 8.4 shows the corresponding log–log refinement plot. The figure confirms the monotonic decrease of the solution differences and supports the conclusion that the solver converges consistently toward the fine-grid reference. In this sense, the test provides useful additional evidence that the full Gray–Scott implementation behaves plausibly under mesh refinement for the nonlinear problem of interest.

Figure 8.4.: Gray–Scott grid-refinement study using the 1024×1024 solution as a numerical reference. The measured L^2 differences in U and V decrease monotonically as the grid is refined.

8.3. Supplementary Verification Checks

In addition to the detailed test cases above, several smaller checks were carried out to support the overall verification strategy. These tests are useful because they isolate specific mathematical or implementation properties, even if they do not require a full standalone discussion.

8.3.1. Constant-Field Laplacian Null Test

As an additional structural check, the discrete Laplacian was applied to a constant field. For a constant function, the continuous Laplacian is identically zero, and the same property is expected from the discrete operator. Therefore, this test serves as a simple but useful sanity check of the assembled diffusion operator.

The test was executed with:

```
r = test_laplacian_constant();
```

For the documented run, the returned result was

$$\texttt{pass} = 1,$$

with

$$\|L\mathbf{1}\|_\infty = 0.000 \times 10^0, \qquad \|L\mathbf{1}\|_2 = 0.000 \times 10^0.$$

Thus, the discrete Laplacian annihilates a constant field exactly in this case. This confirms that the operator satisfies an important basic consistency property and provides additional evidence that the diffusion discretization is assembled correctly.

8.3.2. Symmetry Check of the Discrete Laplacian Matrix

A further structural verification step checks the symmetry of the assembled discrete Laplacian matrix. For the periodic finite-difference diffusion operator used in this project, the Laplacian matrix is expected to be symmetric under the tested setup. This property is important because it supports the correctness of the stencil-to-matrix assembly and helps detect indexing or coefficient-placement errors.

The test was executed with:

```
r = test_laplacian_symmetry();
```

For the documented run, the returned result was

$$\texttt{pass} = 1,$$

and the reported symmetry defect was

$$\|L - L^T\|_\infty = 0.000 \times 10^0.$$

Therefore, the assembled Laplacian matrix is exactly symmetric in this case. This result is fully consistent with the expected structure of the periodic diffusion operator and provides additional evidence that the matrix assembly is implemented correctly.

8.3.3. End-to-End MMS Consistency Check

As a supplementary verification step, the full solver was tested with a manufactured-solution setup. In contrast to the operator-only MMS test of Test Case 1, this check verifies the complete time-dependent workflow, including reaction, diffusion, source-term handling, and explicit Euler time stepping. It is therefore useful as supporting end-to-end verification evidence for the implemented numerical model.

The test was executed with:

```
r = test_mms_endtoend();
```

For the documented run, the solver used a grid

$$N_x = N_y = 128,$$

a time step

$$\Delta t = 0.01,$$

and a final time

$$T = 1.$$

The returned result was

$$\texttt{pass} = 1,$$

and the reported final L^2 errors were

$$\mathrm{err}_U^{L^2} = 1.179 \times 10^{-4}, \qquad \mathrm{err}_V^{L^2} = 5.974 \times 10^{-4}.$$

These errors are small for the chosen grid and time-step size. They are consistent with a correctly implemented manufactured-solution workflow. Figure 8.5 shows the spatial error field for v at the final time. The smooth, structured pattern is typical of discretization error and does not indicate instability or implementation failure.

Overall, this supplementary test provides additional evidence that the complete solver reproduces the manufactured solution with small final errors when the corresponding source terms are included.

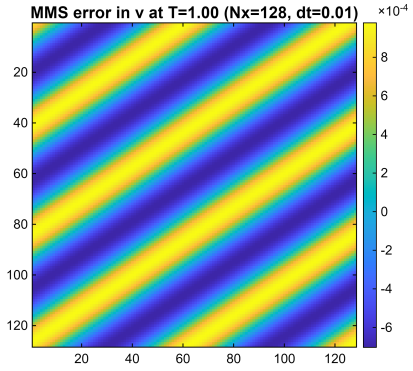

Figure 8.5.: Spatial error field of the manufactured-solution end-to-end test for v at $T = 1$, with $N_x = N_y = 128$ and $\Delta t = 0.01$.

8.3.4. Time-Step Halving Smoke Test

As a quick supplementary consistency check, a simpler time-step halving test was also performed. In contrast to the full temporal convergence study of Test Case 2, this test is not intended as a rigorous order-verification procedure. Instead, it serves as a compact sanity check showing that the numerical solution changes in the expected direction when the time step is refined.

The test was executed with:

```
r = test_timestep_halving_smoke();
```

For the documented run, the solver used a grid

$$N_x = N_y = 128,$$

a final time

$$T = 50,$$

and the baseline time step

$$\Delta t = 0.2.$$

The returned result was

$$\texttt{pass} = 1,$$

and the reported differences between successive refinements were

$$\mathrm{rms}_{12} = 3.706 \times 10^{-5}, \qquad \mathrm{rms}_{24} = 1.851 \times 10^{-5},$$

$$\mathrm{inf}_{12} = 5.798 \times 10^{-4}, \qquad \mathrm{inf}_{24} = 2.891 \times 10^{-4}.$$

The finer-step differences are smaller than the coarser-step differences in both norms:

$$\mathrm{rms}_{24} < \mathrm{rms}_{12}, \qquad \mathrm{inf}_{24} < \mathrm{inf}_{12}.$$

Moreover, the reduction is approximately by a factor of two, which is consistent with the expected first-order behavior of the explicit Euler method under time-step halving.

8.4. Validation-Oriented Pattern Sanity Check

In addition to the formal verification tests presented earlier in this chapter, a long-time Gray–Scott pattern study was carried out as a *validation-oriented sanity check*. The role of this section is intentionally different from that of the preceding test cases. The earlier tests were designed to verify numerical correctness in a strict sense, for example through manufactured solutions, convergence measurements, operator equivalence, boundary-condition enforcement, and consistency checks between different execution paths. By contrast, the present study asks whether the implemented solver also reproduces the kind of plausible and morphologically distinct pattern classes that are commonly associated with the Gray–Scott model in the literature.

The Gray–Scott system is well known for producing a rich variety of self-organized structures depending on the feed and kill parameters. Pearson's classical study established the basic pattern-forming context of the model, while later classification work organized a broader range of visually distinct regimes using labels such as `alpha`, `epsilon`, `kappa`, and `theta` [2, 9]. A modern tutorial-style analysis of the Gray–Scott model and its pattern behavior is also given in [1]. The present test is therefore not intended to replace the formal verification evidence, but rather to complement it by showing that the final implementation produces stable and qualitatively reasonable long-time dynamics for standard Pearson-type parameter choices.

8.4.1. Test Case 8: Pearson-Type Pattern Sanity Check

Aim. This test case checks whether the implemented solver reproduces visually reasonable Gray–Scott patterns for several Pearson-type parameter sets over a long simulation horizon. The aim is not to verify a convergence order or to compare against an exact analytical solution, but to confirm that the final implementation is capable of producing the expected diversity of Gray–Scott morphologies under standard long-time conditions. More specifically, the test examines whether different parameter choices lead to clearly different and stable pattern classes rather than to numerical noise, spurious artifacts, or an incorrect collapse to a trivial state.

Setup. The test uses the Pearson preset on the periodic square domain

$$\Omega = [0, 1) \times [0, 1)$$

with a uniform grid

$$N_x = N_y = 256.$$

The time step is

$$\Delta t = 1,$$

and each case is simulated for

$$N_t = 200000$$

time steps, corresponding to the final time

$$T = 200000.$$

The diffusion coefficients are

$$D_u = 2 \times 10^{-5}, \qquad D_v = 1 \times 10^{-5},$$

and periodic boundary conditions are imposed on all sides.

Four Pearson-type parameter sets are tested:

$$\begin{aligned}
\texttt{alpha}: &\quad F = 0.0075294, \qquad k = 0.04564,\\
\texttt{kappa}: &\quad F = 0.034487, \qquad k = 0.060378,\\
\texttt{epsilon}: &\quad F = 0.019707, \qquad k = 0.055379,\\
\texttt{theta}: &\quad F = 0.040117, \qquad k = 0.060515.
\end{aligned}$$

For each case, snapshots of the U- and V-fields are saved at

$$t = 50000,\ 100000,\ 150000,\ 200000.$$

In the present report, the U-field snapshots at

$$t = 100000$$

are used for comparison, because at this time the morphology of each case is already well developed while still being easy to interpret visually.

How to run the test case. From the project `tests` folder, run:

```
r = test_pattern_sanity_pearson();
```

The test saves the generated images in

```
figures/verification/pattern_sanity/
```

and, for the documented run, completed all four cases successfully. The MATLAB output reported that snapshots were saved for all four parameter sets at

$$t = 50000,\ 100000,\ 150000,\ 200000,$$

and that the full run finished for the cases

$$\texttt{alpha},\ \texttt{kappa},\ \texttt{epsilon},\ \texttt{theta}.$$

Expected result. Since this is a validation-oriented sanity check, the expected result is qualitative rather than pointwise exact. One expects the simulation to produce stable, visually structured Gray–Scott patterns whose morphology depends strongly on the chosen (F, k) pair. In particular, different Pearson-type labels should not all lead to nearly identical images. Instead, some cases should favor spot-dominated structures, while others should exhibit more elongated stripe-like or labyrinthine organizations [2, 9]. The important criterion is therefore not exact agreement with a single published image, but rather the emergence of distinct and plausible pattern classes under long-time integration. As a supporting modern reference, [1] also discusses the sensitivity of Gray–Scott pattern formation to parameter choice.

Obtained result. The test completed successfully, and the returned result was

$$\texttt{pass} = 1.$$

Each of the four parameter sets was simulated over the full

$$200000$$

time steps, and all requested snapshots were saved. The representative U-field snapshots at $t = 100000$, shown in Figures 8.6–8.9, demonstrate that the implemented solver generates clearly different and visually plausible pattern morphologies for the four tested parameter sets.

The `alpha` case in Figure 8.6 is dominated by separated spot-like structures embedded in a mostly high-U background. The spots are irregularly distributed, but they remain localized and well formed. The image does not suggest numerical breakup,

grid-aligned artifacts, or unstable oscillatory contamination. Instead, it shows a coherent spot-dominated regime consistent with the expected Gray–Scott behavior for this type of parameter choice.

The `epsilon` case in Figure 8.7 also belongs to a spot-dominated family, but the pattern is markedly denser and more crowded than in the `alpha` case. Small spot-like depressions in the U-field are distributed much more uniformly across the domain, producing a texture that is finer and more tightly packed. This visual difference is important because it shows that the solver is not merely reproducing one generic spot pattern for all parameter values; rather, it responds sensitively to the change in (F, k) and produces a distinctly different morphological regime.

By contrast, the `kappa` case in Figure 8.8 is no longer spot-dominated. Instead, the solution organizes into broad, elongated, stripe-like structures with a labyrinthine appearance. The stripes are smooth and continuous over long distances, and they form an interconnected network rather than isolated islands. This is a qualitatively different regime from the previous two cases and is visually consistent with the classic Gray–Scott transition from spot patterns to stripe or maze-like patterns under different parameter settings.

The `theta` case in Figure 8.9 is also labyrinth-like, but it is not identical to `kappa`. The stripe network is more intricate and locally more curved, and the overall topology appears somewhat more tangled and fine-scaled. Thus, even within the stripe-dominated family, the implementation still distinguishes between parameter sets and produces visibly different structure. This strengthens the interpretation that the solver is capturing meaningful qualitative pattern dependence rather than generating a single repeated texture.

Taken together, these four cases show two important features. First, the runs remain numerically stable and visually clean over a very long integration horizon. Second, the parameter changes lead to clear morphological differentiation between spot-dominated and labyrinth-dominated regimes, with further variation inside each family. This is exactly the kind of application-level behavior expected from a plausible Gray–Scott implementation [2, 9].

At the same time, the interpretation of this test should remain careful. Because no exact reference field is available, the present result does not prove correctness in the same way as the manufactured-solution or convergence-based tests. Its role is supportive: it shows that the implemented model behaves plausibly, produces stable long-time structures, and reproduces the qualitative diversity associated with Pearson-type Gray–Scott dynamics.

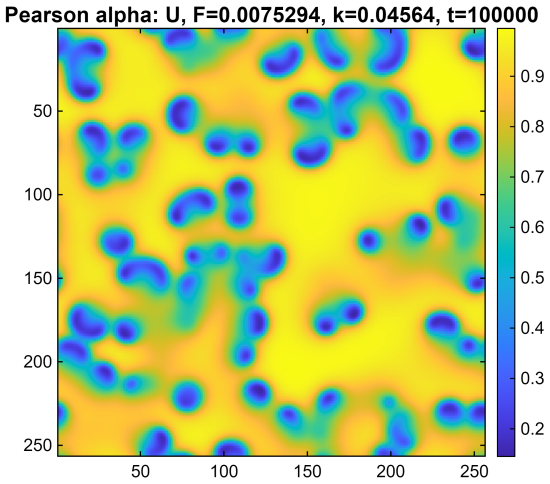

Figure 8.6.: Pearson `alpha` case: representative U-field snapshot at $t = 100000$. The morphology is dominated by separated spot-like structures in a mostly high-U background.

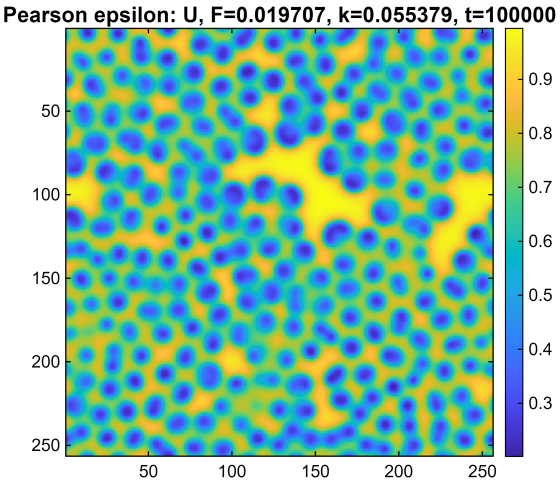

Figure 8.7.: Pearson `epsilon` case: representative U-field snapshot at $t = 100000$. Compared with `alpha`, the pattern is denser and more uniformly populated by small spot-like structures.

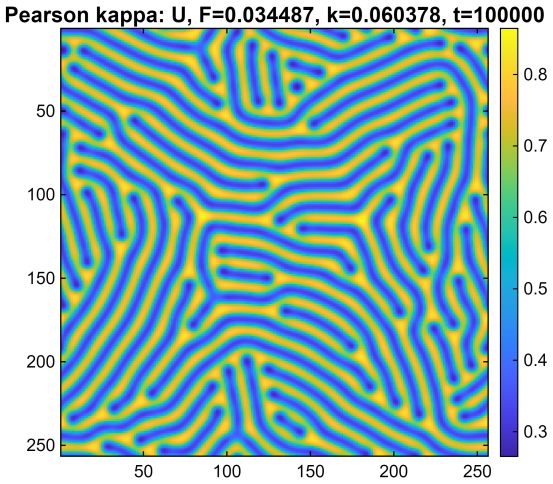

Figure 8.8.: Pearson `kappa` case: representative U-field snapshot at $t = 100000$. The pattern exhibits a broad labyrinth-like stripe morphology rather than isolated spots.

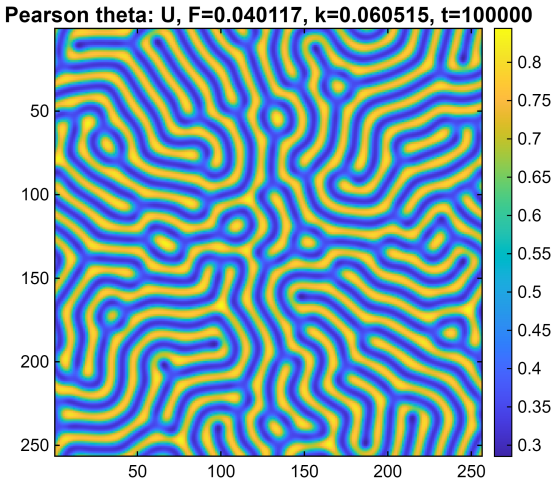

Figure 8.9.: Pearson `theta` case: representative U-field snapshot at $t = 100000$. The pattern is also labyrinth-like, but with a more intricate and locally curved stripe network than in the `kappa` case.

8.5. Verification Summary

Overall, the verification results show that the implemented Gray–Scott solver behaves consistently with the expected numerical properties of the model and its discretization. The manufactured-solution operator test confirmed second-order spatial accuracy of the discrete Laplacian, while the time-step refinement study confirmed the expected first-order behavior of the explicit Euler method. The boundary-condition tests verified robust enforcement of Dirichlet and Neumann conditions, and the matrix–stencil comparison tests showed that the different diffusion implementations are equivalent both at the operator level and in full end-to-end simulations. In addition, the GUI and non-GUI comparison demonstrated that the graphical interface does not alter the numerical results. The supplementary checks further supported the structural correctness and internal consistency of the implementation. Taken together, these results provide strong verification evidence for the numerical core of the project, while the Pearson-type pattern study offers an additional validation-oriented sanity check at the application level.

9. Computation Time Study

This chapter reports an internal timing study of the final program using MATLAB's built-in `tic`/`toc` functions. The measurements were obtained for the stencil-based solver on a 256×256 grid with 20 warm-up steps and 500 timed steps. Two cases were considered: a run without live plotting to isolate the numerical workload, and a run with live plotting every ten steps to assess visualization overhead.

9.1. Function-Level Timing Results

Table 9.1 gives the total runtime of the two representative runs. The version without plotting reflects the computational cost of the numerical core, while the version with plotting shows the additional overhead caused by live visualization.

Table 9.1.: Overall runtime summary for the internal computation-time study.

Case	Setup time [s]	Timed-loop time [s]	Total wall time [s]
No plotting	0.037584	0.892993	0.930577
With plotting	0.037351	8.385918	8.392077

The top-level distribution of runtime over the main program stages is shown in Table 9.2. In the no-plot case, `eulerStep` dominates the execution. In the interactive case, plotting becomes the dominant stage.

Table 9.2.: Top-level function and stage timings for the internal computation-time study.

Stage / Function	No plotting		With plotting	
	Time [s]	Share [%]	Time [s]	Share [%]
eulerStep	0.889539	95.59	1.441456	17.18
plotting	0.000000	0.00	6.937712	82.67
initialCondition	0.024986	2.69	0.003907	0.05
makeOperator	0.003871	0.42	0.000321	0.00
otherOverhead	0.003454	0.37	0.006749	0.08
buildGrid	0.002949	0.32	0.000211	0.00
finalizeParams	0.002364	0.25	0.000528	0.01
defaultParams	0.002224	0.24	0.000594	0.01
buildBCcache	0.001191	0.13	0.000599	0.01

Since `eulerStep` is the dominant numerical stage, its internal composition was also inspected. Table 9.3 shows that `applyDiffusion` is the largest subcomponent inside `eulerStep` in both runs.

Table 9.3.: Internal timing breakdown of `eulerStep`.

Subcomponent	No plotting		With plotting	
	Time [s]	Share [%]	Time [s]	Share [%]
applyDiffusion	0.326522	36.71	0.588390	40.82
applyBoundaryConditions	0.207558	23.33	0.285682	19.82
diagnostics	0.192944	21.69	0.255376	17.72
explicitEulerUpdate	0.066433	7.47	0.141630	9.82
reactionGrayScott	0.063492	7.14	0.088191	6.12
sourceTerm	0.003742	0.42	0.006406	0.44

9.2. Identification of the Computational Bottleneck

The internal timing results lead to a clear bottleneck statement. For the numerical core, the dominant repeated-cost function is `eulerStep`, and the largest internal contributor within it is `applyDiffusion`. When live visualization is enabled, plotting becomes the dominant program-level cost. Thus, the main computational bottleneck of the solver itself is the repeated time-stepping routine, whereas the main practical bottleneck in interactive execution is rendering.

10. Benchmark and Solver Performance Study

10.1. Benchmark Goal and Design

After the internal computation-time study of Chapter 9, a broader benchmark was carried out to compare the practical performance of the implemented diffusion solvers and to support the final solver choice. In contrast to Chapter 9, which analyzed the runtime structure of one representative program run, the present chapter compares the solver variants themselves under the same verified Gray–Scott problem and under controlled benchmark conditions.

The benchmark considers the three implemented diffusion realizations: the dense-matrix formulation, the sparse-matrix formulation, and the stencil-based formulation. The goal is to compare their runtime, live-display behaviour, memory demand, and overall suitability as the default solver of the final program.

10.2. Benchmark Setup

The compared solver modes are `full` (dense matrix), `matrix` (sparse matrix), and `stencil` (matrix-free stencil). Two benchmark modes were used:

- `compute_off`: live plotting disabled, so only the numerical stepping loop is timed,
- `render_on`: live plotting enabled with `plotEvery = 10`, so both numerical work and visualization overhead are included.

The benchmark was carried out on the grid sizes

$$32 \times 32, \quad 64 \times 64, \quad 128 \times 128, \quad 256 \times 256, \quad 512 \times 512.$$

For each benchmark point, 50 warm-up steps were performed before timing, followed by 500 timed steps and 7 repeated measurements. The median runtime was used as the main result.

The time step was solver-independent on each grid. For 32×32, 64×64, and 128×128, the benchmark used $\Delta t = 0.5$. For the finer grids, the time step was reduced according

to the explicit stability restriction, giving $\Delta t = 0.152588$ for 256×256 and $\Delta t = 0.038147$ for 512×512.

The dense solver was included only up to 128×128, because beyond this range its memory cost becomes impractical. Therefore, the main comparison for realistic simulations is between the sparse-matrix and stencil-based solvers.

10.3. Compute-Only Runtime Comparison

Figure 10.1 shows the compute-only runtime results. The dense solver is included only as a small-scale reference and is not competitive beyond the smallest grids. The practically relevant comparison is therefore between the sparse-matrix and stencil-based implementations.

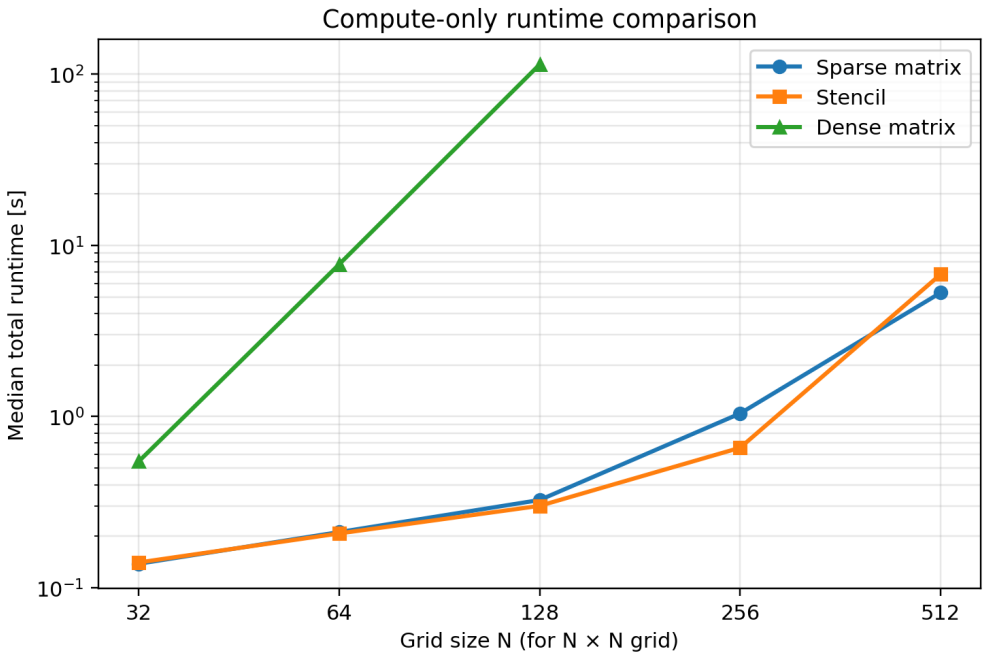

Figure 10.1.: Compute-only benchmark runtime for the three diffusion realizations.

The compute-only results show that neither realistic solver is uniformly fastest on all grids. The stencil solver performs best at 64×64, 128×128, and especially 256×256, while the sparse-matrix solver is slightly better at 32×32 and clearly better at 512×512. Therefore, the final recommendation cannot be based on compute-only runtime alone.

Table 10.1.: Compact compute-only comparison of the sparse-matrix and stencil solvers. Values of matrix/stencil > 1 favour the stencil solver, while values below 1 favour the sparse-matrix solver.

Grid	Matrix [s]	Stencil [s]	matrix/stencil	Faster solver
32×32	0.138	0.140	0.983	Matrix
64×64	0.211	0.208	1.018	Stencil
128×128	0.326	0.301	1.082	Stencil
256×256	1.043	0.658	1.586	Stencil
512×512	5.295	6.777	0.781	Matrix

10.4. Live-Display Runtime Comparison

When live display is enabled, the total runtime includes both numerical work and rendering overhead. The corresponding results are shown in Figure 10.2. Compared with the compute-only case, the differences between the sparse-matrix and stencil solvers become smaller.

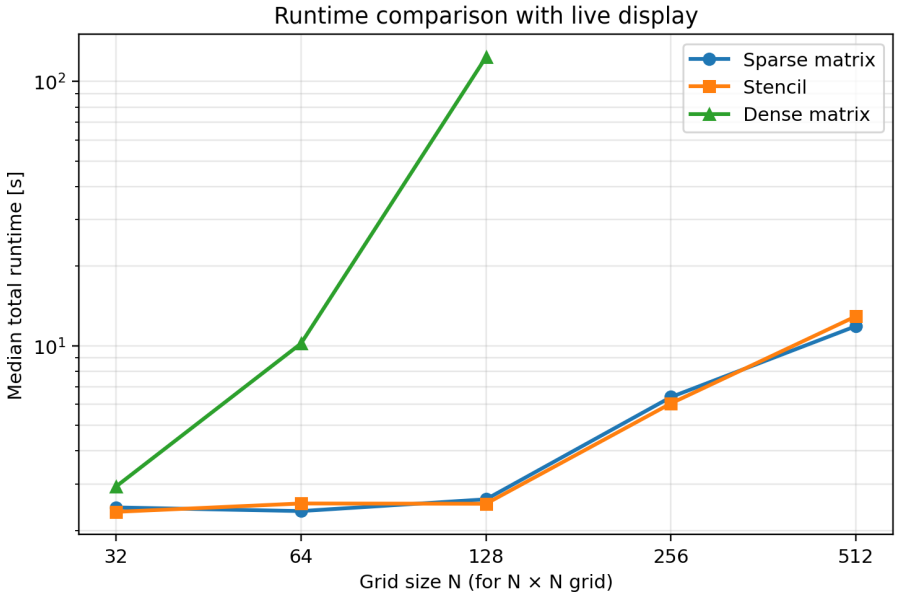

Figure 10.2.: Runtime benchmark with live display enabled.

The achieved median frame rate is shown in Figure 10.3. This gives a more direct view of interactive performance.

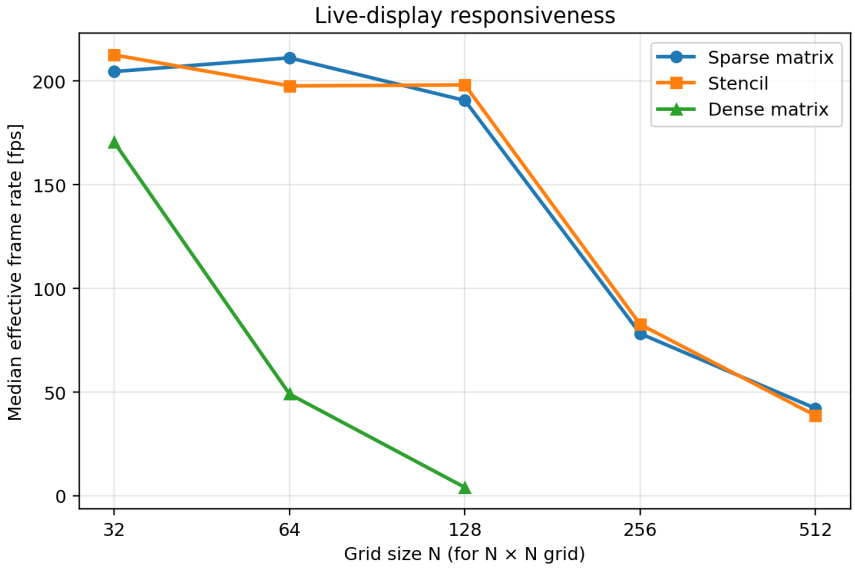

Figure 10.3.: Median frame rate in the live-display benchmark.

Table 10.2 summarizes both the live-display runtime and the median frame rate. The same general pattern appears in both measures: the stencil solver is slightly better at 32×32, 128×128, and 256×256, while the sparse-matrix solver is slightly better at 64×64 and 512×512.

Table 10.2.: Compact live-display comparison of the sparse-matrix and stencil solvers.

Grid	Matrix [s]	Stencil [s]	matrix/stencil	FPS matrix	FPS stencil	Better runtime
32×32	2.445	2.352	1.039	204.5	212.6	Stencil
64×64	2.368	2.530	0.936	211.2	197.6	Matrix
128×128	2.623	2.524	1.039	190.6	198.1	Stencil
256×256	6.395	6.052	1.057	78.2	82.6	Stencil
512×512	11.867	12.927	0.918	42.1	38.7	Matrix

Thus, interactive use does not overturn the basic comparison: both realistic solvers remain close in runtime, and neither dominates absolutely over the whole grid range.

10.5. Memory Footprint Comparison

The measured memory footprint is shown in Figure 10.4. This figure provides one of the clearest distinctions between the solver realizations. The dense solver grows extremely

rapidly and is already impractical at moderate grid sizes. More importantly, the sparse-matrix solver uses much more memory than the stencil solver over the entire tested range.

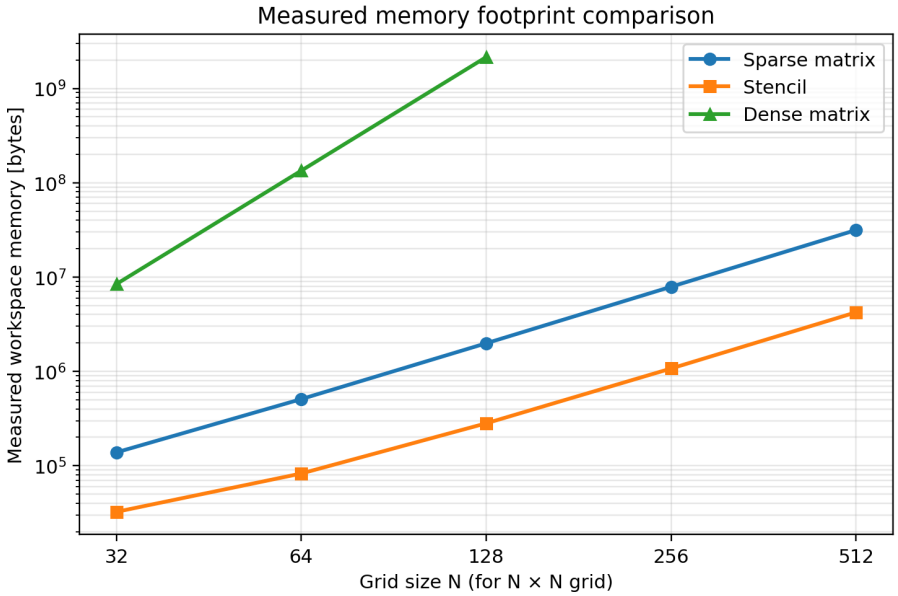

Figure 10.4.: Measured memory footprint of the solver realizations.

Table 10.3 summarizes the measured memory and the relative discrepancy between predicted and measured memory. The memory ratio matrix/stencil grows from about 4.3 on the smallest grid to about 7.5 on the largest tested grid. For the larger grids, the stencil solver also stays much closer to the predicted memory requirement.

Table 10.3.: Measured memory comparison and prediction discrepancy for the sparse-matrix and stencil solvers.

Grid	Matrix [MiB]	Stencil [MiB]	matrix/stencil	Error matrix [%]	Error stencil [%]
32×32	0.132	0.031	4.27	29.86	86.01
64×64	0.484	0.079	6.15	19.24	24.03
128×128	1.893	0.268	7.06	16.47	6.86
256×256	7.522	1.022	7.36	15.72	2.11
512×512	30.029	4.030	7.45	15.50	0.72

The memory results are therefore more one-sided than the runtime results: in all realistic cases, the stencil solver is much more memory-efficient.

10.6. Scaling and Interpretation of Results

A compact summary of the matrix-to-stencil comparison is given in Figure 10.5. The compute-only ratio, the live-display ratio, and the measured-memory ratio are plotted together. This figure shows that the runtime advantage changes with grid size, but the memory advantage consistently remains with the stencil solver.

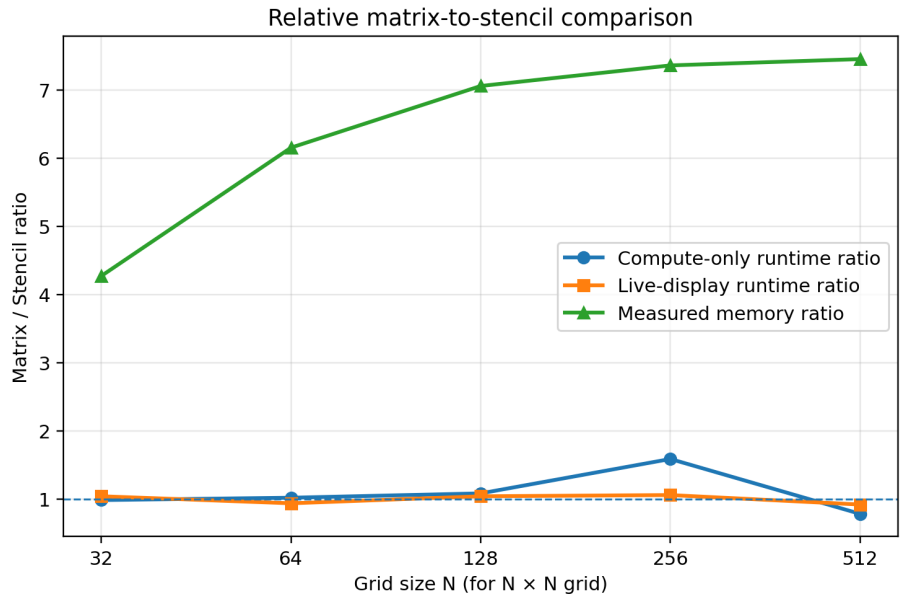

Figure 10.5.: Matrix-to-stencil ratios for compute-only runtime, live-display runtime, and measured memory. Values above 1 favour stencil for runtime ratios and indicate higher matrix memory for the memory ratio.

To assess how reliably the memory behaviour follows the implementation model, Figure 10.6 shows the relative discrepancy between predicted and measured memory. On the larger grids, the stencil solver stays much closer to the predicted memory requirement, whereas the sparse-matrix solver keeps a noticeably larger discrepancy.

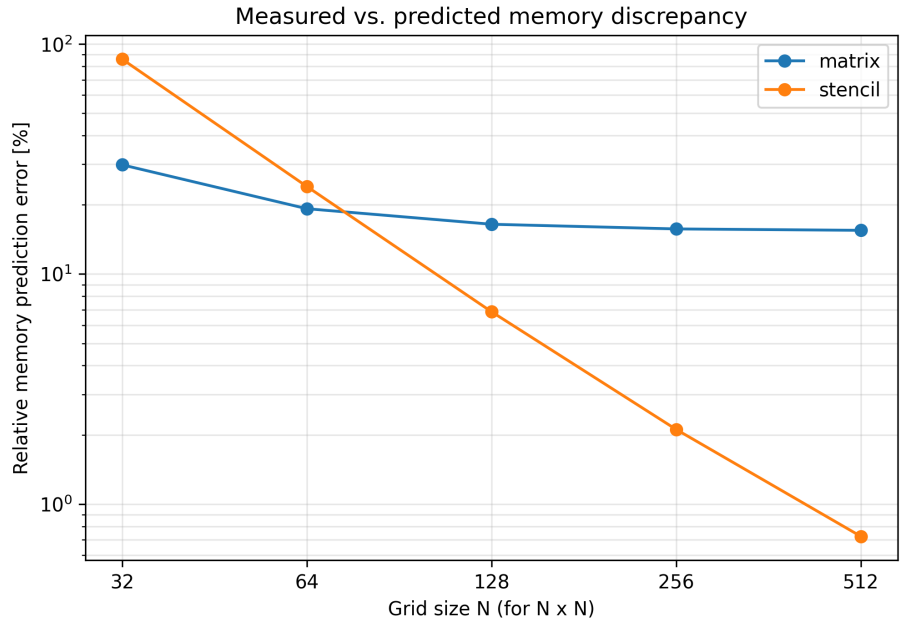

Figure 10.6.: Relative discrepancy between predicted and measured memory for the sparse-matrix and stencil solvers.

Table 10.4 combines the most useful benchmark indicators for the realistic solver pair. It shows clearly that the runtime comparison changes with grid size, whereas the memory comparison consistently favours the stencil implementation.

Table 10.4.: Combined benchmark indicators for the sparse-matrix and stencil solvers.

Grid	Compute ratio	Live-display ratio	Memory ratio	Closer memory prediction
32×32	0.983	1.039	4.27	Matrix
64×64	1.018	0.936	6.15	Matrix
128×128	1.082	1.039	7.06	Stencil
256×256	1.586	1.057	7.36	Stencil
512×512	0.781	0.918	7.45	Stencil

Taken together, the benchmark results lead to three main conclusions. First, the dense solver is not a realistic option beyond small reference cases. Second, the sparse-matrix and stencil solvers are both competitive in runtime, and neither is uniformly fastest on every grid. Third, memory behaviour clearly favours the stencil solver, especially on the larger grids that are most relevant for realistic simulations.

10.7. Selection of the Recommended Default Solver

The final recommended default solver for this project is the stencil-based solver. This recommendation is based on the overall benchmark results of the previous sections. The stencil solver is not always the fastest in absolute runtime, but it remains competitive in both compute-only and live-display execution, while using far less memory than the sparse-matrix solver. In addition, for the larger grids, its measured memory stays much closer to the predicted memory requirement. This makes the stencil implementation more predictable and more suitable for scaling to larger and more complex problems.

Therefore, the stencil solver provides the best overall compromise between speed, memory efficiency, and scalability. For this reason, it was selected as the recommended default solver in the final Gray–Scott program, while the sparse-matrix solver remains a useful alternative and the dense solver is retained only as a small-scale reference implementation.

11. Conclusion

This PPP developed a MATLAB simulation tool for the Gray–Scott reaction–diffusion model together with a structured workflow for testing, benchmarking, and interactive use through a graphical user interface. The final program includes multiple diffusion implementations, configurable boundary conditions, reproducible test cases, and plotting/export functionality, while the GUI and non-GUI workflows share the same numerical core.

The verification results showed that the implementation solves the intended problem correctly. The manufactured-solution test confirmed the expected second-order spatial accuracy of the discrete Laplacian, and the time-step study showed the expected first-order behavior of the explicit Euler scheme. Further tests verified correct boundary-condition treatment, machine-precision agreement between the matrix-based and stencil-based diffusion realizations, and consistency between GUI and non-GUI execution for the documented reference case. A Pearson-type pattern sanity check further supported the physical plausibility of the results.

The main practical question of this project was which solver should be used in the final program. The benchmark study showed that the dense formulation is useful only as a small-scale reference. The sparse-matrix and stencil-based solvers were both competitive in runtime, but the memory results were more decisive: over the relevant grid range, the stencil implementation required less memory and showed better scaling behavior. For this reason, the stencil solver was selected as the default solver of the final program, since it provides the best overall compromise between efficiency and scalability.

The study also showed that repeated time stepping is the dominant numerical cost, while plotting becomes the main overhead in interactive runs. Some limitations remain: the final implementation still uses explicit Euler time integration, so the admissible time step is restricted by stability, and the benchmark was limited to the solver variants and grid sizes considered here. Possible extensions beyond the scope of this PPP include implicit or semi-implicit time stepping, parallelization for HPC or GPU-based computing, and hybrid optimization strategies for the diffusion step. Additional large-scale benchmarks and validation against further literature results would also be useful.

Overall, the project achieved its practical goal: it produced a verified and benchmarked Gray–Scott simulation program in which the stencil-based solver emerged as the most suitable default choice for the final implementation.

Bibliography

[1] Demi L. Gandy and Martin R. Nelson. Analyzing pattern formation in the gray–scott model: An xppaut tutorial. *SIAM Review*, 64(3):728–747, 2022. doi: 10.1137/21M1402868.

[2] J. E. Pearson. Complex patterns in a simple system. *Science*, 261(5118):189–192, 1993. doi: 10.1126/science.261.5118.189.

[3] P. Gray and S. K. Scott. Autocatalytic reactions in the isothermal, continuous stirred tank reactor: Oscillations and instabilities in the system A + 2 B $\rightarrow$ 3 B; B $\rightarrow$ C. *Chemical Engineering Science*, 39(6):1087–1097, 1984. doi: 10.1016/0009-2509(84)87017-7.

[4] Nicholas Sbalbi. Gray-scott-reaction-diffusion. GitHub repository, 2021. URL `https://github.com/nsbalbi/Gray-Scott-Reaction-Diffusion`. MATLAB function for generating and recording Gray–Scott reaction–diffusion models. Accessed: 2026-03-18.

[5] John Burkardt. gray_scott_pde. Web page, 2021. URL `https://people.sc.fsu.edu/~jburkardt/m_src/gray_scott_pde/gray_scott_pde.html`. Gray–Scott PDE code available in MATLAB, Octave, and Python. Accessed: 2026-03-18.

[6] John Burkardt. gray_scott_movie. Web page, 2020. URL `https://people.sc.fsu.edu/~jburkardt/m_src/gray_scott_movie/gray_scott_movie.html`. Gray–Scott movie-generation code available in MATLAB and Python. Accessed: 2026-03-18.

[7] David Augustin, Jakke Neiro, and Thijs van der Plas. Gray-scott-pde. GitHub repository, 2019. URL `https://github.com/SABS-R3-projects/Gray-Scott-PDE`. Notebook-based Python project for solving the Gray–Scott PDE. Accessed: 2026-03-18.

[8] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics, Philadelphia, 2007. doi: 10.1137/1.9780898717839.

[9] Robert P. Munafo. Pearson's classification (extended) of gray-scott system parameters. Web page, 2022. URL `https://www.mrob.com/pub/comp/xmorphia/pearson-classes.html`. Accessed: 2026-03-22.

A. Manual

A.1. Requirements, Project Layout, and Execution Modes

The Gray–Scott project is designed to be executed directly from the root folder of the repository. The main requirement is a working MATLAB installation. In addition, MATLAB App Designer support is needed for the graphical user interface, because the GUI is stored as an `.mlapp` file. No external datasets, third-party libraries, or user-created input files are required for a standard simulation run. The program generates its own initial conditions internally and writes the produced outputs automatically to result folders.

For normal use, the most important files and folders are:

- `run_gui.m`: launcher for the graphical user interface.
- `run_model_class.m`: launcher for one standard non-GUI simulation run.
- `scripts/GrayScottController.mlapp`: App Designer GUI for interactive simulation and visualization.
- `src/`: numerical core of the project, including parameter handling, grid generation, diffusion operators, reaction term, time stepping, model class, and export utilities.
- `tests/`: verification, consistency, convergence, and benchmark scripts.
- `results/`: output folder created automatically during simulations, tests, GUI usage, and benchmark runs.

From the user point of view, the project provides four practical execution modes:

1. **GUI mode:** interactive use through the App Designer interface for manual exploration of the model.
2. **Non-GUI mode:** scripted execution through `run_model_class.m` for reproducible runs.
3. **Test mode:** execution of the main verification suite. The entry script is `tests/run_all_tests.m`.
4. **Benchmark mode:** performance comparison of solver variants. The entry script is `tests/benchmark_solvers.m`.

The project does not use external parameter files such as JSON, TXT, CSV, or XML files for standard runs. Instead, the simulation settings are stored in a MATLAB structure named `p`. In non-GUI mode, this structure is created directly in the launcher script. In GUI mode, the same parameter structure is managed internally through the interface controls. Thus, the effective input format of the program is a MATLAB parameter structure rather than a separate configuration file.

In the final project version, the stencil-based diffusion solver is the default choice for normal execution. The matrix-based and full-array variants remain available mainly for verification and benchmarking.

The following sections explain how to use the GUI, how to run the model without the GUI, how to execute the standard tests and the benchmark script, and where the generated outputs are stored.

A.2. How to Run the GUI

The graphical user interface is the most convenient execution mode for interactive exploration of the Gray–Scott model. To start it, open the project root folder in MATLAB and run

```
run_gui
```

This command adds the required source folders to the MATLAB path and launches the App Designer application `GrayScottController`. The GUI is not a separate implementation of the model. It uses the same numerical core as the scripted workflow, but provides an interactive front end for changing settings and visualizing the results.

At startup, the GUI creates a default Gray–Scott model automatically and opens a separate plot window. In the final project version, the GUI uses the stencil-based diffusion solver as its normal execution mode.

The main user-controlled settings in the GUI are:

- **Run control:** `Run`, `Pause`, `Reset`, and `Restart`.
- **Reaction parameters:** the sliders for `F` and `k`.
- **Displayed field:** selection of the plotted concentration field `U` or `V`.
- **Boundary conditions:** `Periodic`, `Neumann`, or `Dirichlet`.
- **Visual options:** colormap and plotting appearance.
- **Recording options:** video frame rate and video format.

- **Pattern presets:** predefined `F-k` combinations for characteristic pattern regimes.

The diffusion coefficients `D_u` and `D_v` are shown in the interface for transparency, but in the final GUI they are read-only. Therefore, the GUI is mainly intended for interactive study of pattern formation through the parameters `F` and `k`, while more detailed changes are better handled in the scripted non-GUI workflow.

A typical GUI workflow is as follows:

1. Start the application with `run_gui`.
2. Select the field to display, usually `V`, because it shows the pattern structure clearly.
3. Adjust `F` and `k` manually or choose a preset from the menu.
4. Select the desired boundary-condition type.
5. Press `Run` to start the simulation and `Pause` to stop it temporarily.
6. Use `Restart` if the simulation should begin again with the current `F` and `k` values.
7. Use `Reset` if the GUI should return to its original default values and rebuild the initial state.

The difference between `Reset` and `Restart` is important. `Restart` keeps the currently selected reaction parameters and rebuilds the model from the beginning using those values. `Reset`, in contrast, restores the original GUI defaults and then rebuilds the model. This allows the user either to repeat the current configuration or to return quickly to the startup configuration.

The GUI also supports model storage through the `File` menu. The entry `Save model` stores the current model object as a MATLAB `.mat` file, and `Load model` reloads such a saved model. Video recording can also be activated directly in the GUI. When recording is enabled, the application writes a video file automatically to the `results/` folder.

As a simple example, suppose that the user wants to inspect a spot-forming regime visually. A practical procedure is to start the GUI, select the `V` field, choose an appropriate spot-like preset from the menu, and press `Run`. If the generated pattern is interesting, recording can be activated before the next run so that the temporal evolution is saved automatically. If the same pattern should then be compared under a different boundary condition, the boundary-condition type can be changed and the model can be started again from the initial state.

The GUI is therefore best suited for exploratory work, visual comparison of parameter choices, video recording, and demonstration of the model during presentations. For reproducible scripted runs with explicit parameter editing, the non-GUI workflow is more appropriate.

A.3. How to Run the Model Without the GUI

The non-GUI workflow is intended for reproducible scripted simulations. It is the most suitable execution mode when parameter values should be documented explicitly, when a run should be repeated under controlled conditions, or when output files should be generated in an organized run folder.

To execute one standard simulation without the graphical user interface, open the root folder of the project in MATLAB and run

```
run_model_class
```

The script creates the parameter structure `p`, normalizes it, builds the computational grid, initializes the `GrayScottModel` object, performs the time-stepping loop, optionally updates a live figure, optionally exports PNG snapshots, and finally stores the last simulation state.

Basic workflow

The normal workflow of `run_model_class.m` is:

1. add the `src/` folder to the MATLAB path,
2. create a default parameter structure `p`,
3. select the diffusion mode,
4. call `finalizeParams(p)` to normalize and validate the settings,
5. create a timestamp-based output folder inside `results/`,
6. run the simulation,
7. save the main outputs of the run.

In the final project version, the script uses the stencil solver as the default solver. Thus, a normal call of `run_model_class` already starts from the recommended standard execution mode.

Parameter format

The non-GUI launcher does not read a separate external parameter file. Instead, all simulation inputs are defined directly in MATLAB through the structure `p`. The most important fields are:

- `Du`, `Dv`, `F`, `k`: Gray–Scott model parameters,
- `Nx`, `Ny`, `Lx`, `Ly`: grid and domain parameters,
- `dt`, `T`: time-step size and final time,
- `seed`, `icType`, `icPerturb`: initial-condition settings,
- `diffusionMode`: solver selection,
- `plotField`, `plotEvery`, `savePngEvery`: output control,
- `BC`: per-side boundary-condition structure.

Some of these fields use fixed keywords that must be written exactly:

- `diffusionMode`: `"matrix"`, `"stencil"`, or `"full"`,
- `plotField`: `"u"` or `"v"`,
- `p.BC.<side>.type`: `"periodic"`, `"dirichlet"`, or `"neumann"`,
- `defaultParams(...)` presets: `"baseline"` or `"pearson"`.

The call to `finalizeParams(p)` is important because it checks and normalizes these entries before the simulation starts. If a keyword is written incorrectly, the script stops with an error instead of running with an inconsistent setup.

Important execution rule

The script computes the number of time steps as `Nt = T / dt`. Therefore, the chosen values of `T` and `dt` must be compatible such that `T/dt` is an integer. Otherwise the script stops with an error. When parameters are modified manually, this condition should always be checked.

Simple examples

Example 1: standard run. For a default run with the standard stencil solver, no manual changes are needed. Open the project root folder and execute

```
run_model_class
```

This uses the default parameter structure, runs one simulation, and writes the outputs automatically into a newly created subfolder of `results/`.

Example 2: change the preset and output settings. A user may want to start from the Pearson-style preset, visualize the `u`-field, and export images less frequently. In that case, the beginning of `run_model_class.m` can be edited as follows:

```
p = defaultParams("pearson");
p.diffusionMode = "stencil";
p.plotField = "u";
p.plotEvery = 50;
p.savePngEvery = 100;
p = finalizeParams(p);
```

After saving the file, run

```
run_model_class
```

The simulation will then use the Pearson-type parameter set and export snapshots of the selected field according to the specified output cadence.

Example 3: modify one model parameter and the boundary condition. To compare the influence of a different feed rate and a different boundary-condition type, the parameter block can be edited, for example, to

```
p = defaultParams();
p.diffusionMode = "stencil";
p.F = 0.022;
p.k = 0.051;

p.BC.left.type   = "neumann";
p.BC.right.type  = "neumann";
p.BC.bottom.type = "neumann";
p.BC.top.type    = "neumann";

p = finalizeParams(p);
```

This produces a run with modified reaction parameters and homogeneous Neumann conditions on all four boundaries.

Plotting and PNG export

The non-GUI script distinguishes between live plotting and image export:

- `plotEvery` controls how often the MATLAB figure is updated during the run,
- `savePngEvery` controls how often PNG files are written to disk.

A value of `0` disables the corresponding feature. For example, setting

```
p.plotEvery = 0;
p.savePngEvery = 0;
```

disables intermediate live plotting and intermediate PNG export. The script still saves the final state and exports one final snapshot at the end of the run.

A.4. How to Run the Tests

The project contains a structured verification folder for automated checks of the numerical implementation. These tests are intended to confirm that the solver behaves correctly with respect to operator properties, boundary-condition handling, manufactured-solution verification, time-step behavior, and agreement between different diffusion implementations.

In the final project version, the standard test entry point is

```
tests/run_all_tests
```

If MATLAB is already opened inside the `tests/` folder, the same suite can also be started with

```
run_all_tests
```

The script adds the required project paths automatically, executes the selected verification tests one after another, prints a pass/fail summary in the MATLAB command window, and stores a summary file under `results/tests/`.

Standard verification suite

The standard test runner contains the important fast-to-moderate checks that should be executed routinely. In the final repository, these are:

- `test_laplacian_constant`: checks that the Laplacian behaves correctly for a constant field.
- `test_laplacian_symmetry`: checks the expected symmetry property of the discrete operator.
- `test_boundary_conditions`: verifies boundary-condition handling.
- `test_timestep_halving_smoke`: short smoke test for time-step refinement behavior.
- `test_mms_endtoend`: manufactured-solution verification of the full solver workflow.
- `test_mms_operator_convergence`: convergence check for the discrete operator.
- `test_timestep_convergence`: verification of the expected time-step behavior.
- `test_laplacian_stencil_equivalence`: agreement check between stencil and matrix-style Laplacian application.
- `test_matrix_stencil_endtoend_equivalence`
  end-to-end comparison between matrix and stencil solver paths.

This curated set is the recommended standard verification procedure for routine use because it covers the essential numerical properties without including the longest validation scripts.

Long or special tests to run separately

Not every script in the `tests/` folder belongs to the standard routine suite. Some checks are intentionally kept separate because they are slower, more qualitative, or intended for special validation tasks.

1. Pearson pattern sanity test

The script

```
tests/test_pattern_sanity_pearson
```

is a long qualitative validation run based on Pearson-style cases. It is intended for visual sanity checking of the produced patterns rather than for fast routine testing.

2. Grid-refinement study

The script

```
tests/test_grayscott_grid_refinement
```

performs a dedicated grid-refinement study. Because it is more expensive than the routine checks and is used for a specific numerical study, it is better treated as a standalone verification script.

3. GUI versus non-GUI consistency check

The folder `tests/gui_consistency/` contains a separate workflow to verify that the GUI reproduces the same numerical results as the non-GUI execution path. This check is not part of `run_all_tests`, because it requires a dedicated comparison workflow. The recommended procedure is:

```
ref = run_nongui_reference_dump();
gui = run_gui_consistency_dump();
T   = compare_gui_nongui(ref.refFile, gui.guiFile);
```

This workflow generates reference dumps, compares intermediate and final states, and saves comparison outputs for later inspection.

For ordinary development and routine verification, the standard call to `run_all_tests` is sufficient and recommended. The standalone scripts should be used only when their specific purpose is needed.

A.5. How to Run the Benchmarks

The benchmark scripts are intended for systematic performance comparison of the available diffusion implementations. In the final project version, the main benchmark entry point is the MATLAB function `benchmark_solvers()`, which compares the sparse-matrix solver, the stencil-based solver, and the dense/full solver under a fixed benchmark protocol. The benchmark is not a verification test in the strict sense; instead, it measures runtime and memory-related quantities for the different implementations under controlled conditions.

To run the benchmark from MATLAB, a practical procedure from the project root folder is

```
addpath(genpath(fullfile(pwd,'tests')));
benchmark_solvers
```

The benchmark script itself adds the required source folders automatically, so the main purpose of the `addpath` command above is to make the benchmark function callable from the project root. After the run finishes, MATLAB writes the results into a new timestamp-based benchmark folder under `results/`.

What the benchmark measures

The benchmark compares three diffusion realizations:

- `matrix`: sparse Laplacian matrix multiplication,
- `stencil`: matrix-free stencil diffusion,
- `full`: dense/full Laplacian matrix multiplication.

It evaluates these solvers in two timing modes:

- `compute_off`: solver runtime without live visualization,
- `render_on`: solver runtime with live display updates.

The timed region contains only the stepping loop. Grid construction, operator setup, initial-condition preparation, and warm-up steps are excluded from the timed interval. This makes the benchmark suitable for comparing the solver cost itself rather than the total startup overhead.

Default benchmark setup

In the final repository, the benchmark uses the following default configuration:

- grid sizes: `32x32`, `64x64`, `128x128`, `256x256`, and `512x512`,
- dense/full solver only on `32x32`, `64x64`, and `128x128`,
- warm-up steps: `50`,
- timed steps: `500`,
- repetitions per case: `7`,
- random seed: `1`,
- live-display update interval in render mode: every `10` steps.

These settings are already encoded in the final benchmark script and normally do not need to be changed for routine use.

Simple benchmark example

A standard benchmark run consists of the following MATLAB commands:

```
addpath(genpath(fullfile(pwd,'tests')));
benchmark_solvers
```

After the computation finishes, inspect the newest benchmark folder inside `results/`. It contains the main benchmark data files, including a MATLAB data file and a CSV table for later analysis.

Interpretation note

The benchmark is designed specifically to compare diffusion implementations under a common numerical setup. The diffusion operators use periodic connectivity internally, while physical boundary conditions such as Dirichlet or Neumann are enforced after each time step rather than being part of the benchmarked operator itself. Therefore, the benchmark should be interpreted as a solver-performance study and not as a comparison of different physical boundary-condition models.

A.6. Output Folders and Generated Files

The project writes its generated data automatically into output folders, so the user normally does not need to create result directories manually. Most outputs are stored under the folder `results/`. The exact files depend on the selected execution mode.

Outputs of a non-GUI simulation run

When `run_model_class` is executed, the script creates a new timestamp-based subfolder inside `results/`. Its structure has the form

```
results/<timestamp>_<caseName>/
```

This folder collects the outputs of exactly one scripted simulation run. The most important generated files are:

- `meta.mat`: MATLAB data file containing metadata of the run, including the parameter structure.
- `log.txt`: text summary of the most important run settings.
- PNG image files: exported snapshots of the selected field during the run and at the final state.
- `final_state.mat`: MATLAB file containing the final numerical state of the simulation.

These files make the non-GUI workflow reproducible and suitable for later inspection.

Outputs of the GUI

The GUI also writes files automatically into the `results/` folder when save or recording operations are used. In practical use, the most relevant GUI-generated outputs are:

- saved model files in MATLAB `.mat` format,
- recorded video files of simulation runs.

The saved model files are useful when a simulation state should be reopened later from the GUI. The video files are produced when recording is enabled in the interface.

Outputs of the standard test suite

The standard verification suite started by

```
tests/run_all_tests
```

stores its collected summary in

```
results/tests/results_summary.mat
```

This file contains the returned result structures of the executed routine tests. The command window output during the test run provides the immediate pass/fail feedback, while `results_summary.mat` stores the structured data for later use.

Longer standalone validation scripts may generate their own outputs depending on the specific workflow. For example, dedicated GUI consistency checks create separate comparison data for inspection.

Outputs of the benchmark workflow

The benchmark function creates a separate timestamp-based output folder of the form

`results/benchmarks_<timestamp>/`

Inside this folder, the main generated files are:

- `benchmark_results.mat`: raw MATLAB benchmark data,
- `benchmark_table.csv`: tabulated benchmark summary.

The MATLAB file is useful for later detailed analysis in MATLAB, while the CSV file is convenient for inspection, table generation, or further processing.

Practical interpretation of file formats

The generated file formats in the project have the following practical roles:

- `.mat`: MATLAB-native storage of states, models, metadata, and structured results.
- `.txt`: human-readable summaries such as run logs.
- `.png`: image export for simulation snapshots.
- video files: recorded GUI animations for presentation and visual inspection.
- `.csv`: tabulated benchmark data for easy comparison and export.

Thus, the project does not require external input data files for normal execution, but it does produce several well-defined output formats for analysis, visualization, and documentation.

B. Git Log

This appendix contains the Git commit history of the Gray–Scott code repository used in this PPP.

```
commit 0a87f142bdd83988f54422d73bdb891dbad79182
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-03-24

    Add README file for tests, make tests run independently,
    ↪  fix the legend problem in plotting

commit 243cf62ccbdeeff869b1a06035fe22b800d6bd4f
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-03-18

    Clean up final project defaults, update tests, and make
    ↪  stencil the default solver

commit 88c0008f307f10ba852e18c4897056c22a3748b1
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-03-17

    Replace process memory metric with whos-based solver memory
    ↪  and add benchmark report plots, and some bugs fixed

commit 06abe6d4bebc19e9583b926128979c6df3abaa25
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-03-16

    Some minor bugs fixed.

commit eafb52a6ca335671a6394d339a43c293ace810bb
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-03-12

    Finalize tests and verification scripts
```

```
    Update tests to use new eulerStep(op,p,t) API, fix Pearson
     ↪ pattern initialization, make plotting scripts
     ↪ path-independent, and stabilize MMS, timestep, and
     ↪ boundary condition verification utilities.

commit 63ecad007d80e73ccd9ee1884c629de4689aacac
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-03-11

    Improve controller robustness and runtime safety: add
     ↪ rootDir-based project paths, move logs and outputs to
     ↪ results/, implement safe GUI shutdown with isClosing,
     ↪ fix video format logic and writer cleanup, centralize
     ↪ status bar updates, validate loaded models
     ↪ (GrayScottModel), improve run-loop stability, and
     ↪ clarify plot titles after reset/restart/BC changes.

commit eff16d3ae39e99d27a3e2b23f2a497c9ae097bb8
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-26

    Finalize entry points and GUI lifecycle for submission:
     ↪ move run_model_class to root, add root-safe run_gui
     ↪ launcher, fix App Designer close lifecycle (no zombie
     ↪ singleton, no DestroyedObject errors), make recording
     ↪ path root-safe, remove dead solver/video state,
     ↪ centralize status updates, and harden plot window
     ↪ shutdown.

commit 7763f1430b535eaac3252e871ca625f50f099e4b
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-26

    Harden I/O layer: path-safe exportFrame, resilient
     ↪ plotState, validated benchmark plotting

commit 2b3d7c61fdbdcbf41046eb68d479fe732b5810b2
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-24

    refactor: remove legacy solver interface

    - Deleted p.solver and all mapping logic
    - Standardized on p.diffusionMode (matrix/stencil/full)
```

```
    - Updated benchmarks, plotting, tests, and scripts
    - Removed legacy CSV compatibility
    - Verified no remaining solver references
    - Passed end-to-end smoke tests

commit ea1547cc4d5b1f03175d7a9aa4f0bd5182b80fae
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-23

    Simplify GUI: reset F/k, BC reset, status bar, recording
     ↪  lamp

commit 5824f38b0ea77af510dfbed455129bcc0b08eac3
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-21

    Add boundary condition support, update core/operators,
     ↪  refresh GUI and report assets

commit ba6f11330608aceed9d127bc7458a08d80a0160d
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-20

    Some minor errors is handeled

commit 0531127479ad96898b98cf5dcb71c48d0a4780d4
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-20

    The Record function is added

commit e006dc3f0b7b53e16676865234695f00bc16d572
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-18

    Add App Designer GUI controller and update GrayScottModel
     ↪  implementation

commit 6fb4ee1d8481841f12da6340078180c75a9ec7e9
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-13
```

```
    Refactor project structure: separate model, operators,
     ↪  core, and io modules; explicit solver abstraction
     ↪  (sparse/dense/stencil)

commit 551193f1e302d96d6c89fe5e4e375f7d368dae36
Author: MehdiBakhshiZadeh <mehdi.bakhshi02@gmail.com>
Date:   2026-02-13

    Initial clean commit: Gray-Scott model implementation with
     ↪  tests and baseline variant
```